

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
"ВЫСШАЯ ШКОЛА ЭКОНОМИКИ"
ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ НАУК**

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

**ГЕНЕРАЦИЯ ПЕРСОНАЛИЗИРОВАННЫХ УЧЕБНЫХ КОНСПЕКТОВ ПО
ПРЕДМЕТАМ ЕГЭ НА ОСНОВЕ ЯЗЫКОВЫХ МОДЕЛЕЙ И
ПОЛЬЗОВАТЕЛЬСКОЙ АНАЛИТИКИ**

**GENERATION OF PERSONALIZED EDUCATIONAL NOTES FOR UNIFIED
STATE EXAM SUBJECTS BASED ON LANGUAGE MODELS AND USER
ANALYTICS**

по направлению подготовки 01.04.02 Прикладная математика и информатика
образовательная программа «Искусственный интеллект»

Выполнили студенты:

Ковыркин Денис Андреевич

Низов Александр Сергеевич

Руководитель ВКР:

Доцент ФКН НИУ ВШЭ

Кантонистова Елена Олеговна

Соруководитель ВКР:

Преподаватель ФКН НИУ ВШЭ

Ахтямов Павел Ибрагимович

Москва 2026

Содержание

Аннотация	3
Введение	4
1 Обзор литературы	5
1.1 Персонализация обучения	5
1.2 Генерация учебных материалов	6
1.3 Существующие решения и методы оценки качества	8
2 Архитектура системы и подход к реализации	10
2.1 Общая архитектура системы	10
2.2 Генерация текстового содержания конспекта	10
2.3 Генерация визуализаций	11
2.4 Сценарии использования и контур развёртывания	13
2.5 Оценка качества	13
3 Генерация визуализаций: реализация и эксперименты	15
3.1 Постановка задачи	15
3.2 Архитектура программного контура	16
3.3 Промежуточный формат описания сцены	19
3.4 Двухпроходный планировщик визуалов	22
3.5 Конвертация плана в описание сцены	25
3.6 Движок векторного рендеринга и режимы отказоустойчивости	26
3.7 Первая интеграционная проверка на корпусе из четырёх конспектов	28
3.8 Расширение в сторону пошаговой геометрии	34
3.9 Стереометрия и автоматическое определение видимости рёбер	38
3.10 Эволюция системы: итерации улучшений	41
3.11 Количественные эксперименты и анализ предельных случаев	47
3.12 Ограничения и направления развития	52
3.13 Выводы по главе	61
Заключение	63
Список литературы	68

Аннотация

Подготовка к единому государственному экзамену по математике требует учебных материалов, адаптированных под индивидуальный уровень каждого ученика. Существующие платформы либо предлагают адаптивный подбор задач без генерации объяснительного контента, либо работают в диалоговом формате без структурированных визуальных материалов. Ни одна из них не закрывает полный цикл: от аналитики знаний ученика к персонализированному конспекту с математическими иллюстрациями.

В данной работе разработан модуль генерации таких конспектов с применением больших языковых моделей. На вход поступает профиль компетенций ученика, построенный по истории решённых задач и допущенных ошибок; на выходе — структурированный конспект с текстом и визуальными материалами, адаптированными под уровень конкретного школьника. Отдельный пайплайн отвечает за автоматическое создание математических иллюстраций: графиков функций, геометрических чертежей и блок-схем, — обеспечивая их геометрическую корректность.

Ключевые слова: персонализированное обучение, генерация учебных материалов, большие языковые модели, генерация с извлечением информации

Abstract

Preparing for the Unified State Exam in mathematics requires learning materials tailored to each student's individual knowledge level. Existing platforms either offer adaptive task selection without generating explanatory content, or operate in a dialogue format without structured visual materials. None of them covers the full cycle from student knowledge analytics to a personalized study note with mathematical illustrations.

This work presents a module that generates such study notes using large language models. The input is a student competency profile built from the history of solved problems and errors; the output is a structured note with text and visual materials adapted to the individual student's level. A dedicated pipeline handles the automatic generation of mathematical illustrations — function graphs, geometric diagrams, and flowcharts — ensuring their geometric correctness.

Keywords: personalized learning, educational content generation, large language models, retrieval-augmented generation

Введение

Настоящая работа выполняется в рамках группового проекта по созданию платформы персонализированной подготовки к ЕГЭ по математике. Платформа строится на пользовательской аналитике: из истории решённых задач формируется вектор компетенций ученика по темам кодификатора ФИПИ, который затем управляет генерацией учебного контента. Фокусом данной работы является модуль генерации конспектов с визуальными материалами.

Типичный конспект по математике без иллюстраций неполноценен для ряда тем ЕГЭ. Задачи на графики функций, взаимное расположение геометрических фигур или анализ производной по графику требуют не просто словесного описания, а корректного визуального представления. Использование моделей генерации изображений по текстовому описанию для этой цели неприемлемо: такие модели систематически нарушают точные геометрические отношения и некорректно воспроизводят подписи и формулы [3, 25]. В данной работе предлагается двухшаговый процесс: языковая модель определяет структуру конспекта и места для визуальных элементов, формирует их текстовые описания, затем отдельный модуль на основе этих описаний генерирует JSON-спецификацию, а специализированный движок рендеринга строит по ней изображение в формате масштабируемой векторной графики (Scalable Vector Graphics, SVG). Такой подход обеспечивает геометрическую корректность, валидируемость выхода и возможность адаптировать сложность иллюстрации под уровень ученика.

Цель работы — разработать и реализовать модуль генерации персонализированных учебных конспектов по математике ЕГЭ, включающих корректные визуальные материалы, на основе пользовательской аналитики и языковых моделей.

Для достижения этой цели в работе решаются следующие задачи.

- 1 Проанализировать существующие подходы к персонализации учебного контента, генерации учебных материалов и автоматическому созданию математических визуализаций.
- 2 Спроектировать архитектуру процесса генерации конспектов: от вектора компетенций ученика к структурированному тексту с визуальными элементами.
- 3 Разработать схему JSON-представления для математических визуализаций и реализовать движок SVG-рендеринга на её основе.
- 4 Интегрировать модуль визуализаций в общий процесс генерации конспектов с учётом профиля ученика.

5 Оценить качество генерируемых конспектов.

Разработанный модуль встраивается в действующую платформу подготовки к ЕГЭ «Прорыв» [30] и обеспечивает автоматическое создание персонализированных учебных материалов для реальных пользователей. Предложенный подход к структурированной генерации визуализаций через промежуточное JSON-представление применим и для других предметов, где требуются точные графические иллюстрации.

1 Обзор литературы

1.1 Персонализация обучения

Учащиеся, готовящиеся к ЕГЭ по математике, существенно различаются по исходному уровню знаний, темпу усвоения и характерным ошибкам: одни испытывают трудности с алгеброй и функциями, другие систематически ошибаются в планиметрии, третьи не справляются с задачами на параметры. Универсальный конспект, рассчитанный на «среднего» ученика, не учитывает эту неоднородность. Для сильного ученика такой конспект окажется избыточным: он будет тратить время на повторение уже освоенного материала. Для слабого, напротив, он может быть недостаточным, поскольку пропускает промежуточные шаги рассуждения. Отсюда возникает задача персонализации: учебный материал должен адаптироваться к текущему состоянию знаний конкретного школьника.

Педагогическое обоснование такого подхода представлено в работе Блума [2], показавшего, что индивидуальное тьюторство повышает результаты обучения на два стандартных отклонения по сравнению с групповым форматом. Интеллектуальные обучающие системы (ИОС) пытаются воспроизвести этот эффект программными средствами. Мета-анализ VanLehn [23] показал, что ИОС с пошаговой обратной связью приближаются по эффективности к человеческому тьюторству, а Steenbergen-Hu и Cooper [21] подтвердили положительный эффект ИОС на успеваемость именно по математике в школьном сегменте (K–12).

Вместе с тем эффективность персонализации критически зависит от качества диагностики. Если система неточно оценивает, какие темы ученик уже освоил, адаптация содержания может не только не помочь, но и навредить: например, пропустить критически важное объяснение или, наоборот, перегрузить ученика уже знакомым материалом. Обзор адаптивных платформ [22] систематизирует основные барьеры: недостаточную интерпретируемость моделей ученика, сложность интеграции в образовательный процесс и риск подмены обучения выдачей готовых ответов. Последний аспект особенно актуален для систем на основе

генеративных моделей.

Формализованную модель компетенций ученика обеспечивает семейство методов отслеживания знаний (Knowledge Tracing, KT). Классический подход — байесовское отслеживание знаний (Bayesian Knowledge Tracing, BKT) [6] — моделирует вероятность освоения навыка как скрытую переменную марковской модели. BKT прост в интерпретации, однако предполагает независимость навыков. Нейросетевые методы, прежде всего глубокое отслеживание знаний (Deep Knowledge Tracing, DKT) [18], позволяют моделировать зависимости между навыками, но ценой снижения интерпретируемости. Обзор [20] систематизирует современные подходы к KT, а Lu et al. [16] развивают направление, в котором акцент смещается с предсказания ответа на диагностику причин затруднений, что ближе к задаче генерации адресного конспекта.

В данной работе отслеживание знаний выполняет конкретную инженерную функцию: превращает историю взаимодействий пользователя с платформой в вектор компетенций по темам ЕГЭ. Пространство тем основано на кодификаторе ФИПИ [35], что обеспечивает привязку модели к реальной структуре экзамена. Такая привязка принципиальна: без неё вектор компетенций оставался бы абстрактной конструкцией, не соотнесённой с тем, что именно проверяется на экзамене. Полученный вектор становится управляющим сигналом для генерации конспекта: определяет, какие темы включить, какой уровень детализации выбрать, где добавить примеры и визуальные пояснения. В следующем разделе рассматриваются методы генерации учебного контента, использующие такой вектор в качестве входного параметра.

1.2 Генерация учебных материалов

Вектор компетенций, описанный в предыдущем разделе, становится полезным только при наличии механизма, который преобразует его в учебный материал. В данном разделе рассматриваются три компонента такого механизма: генерация текста на основе больших языковых моделей (Large Language Model, LLM), обеспечение фактической точности через генерацию с извлечением информации (Retrieval-Augmented Generation, RAG) и автоматическое создание визуальных материалов.

Обзор [29] систематизирует сценарии применения LLM в образовании, выделяя роли модели (помощник ученика, помощник преподавателя, компонент платформы) и фиксируя основные риски: галлюцинации, фактические ошибки, подмену обучения выдачей готового ответа. Для данной работы принципиально, что конспект отличается от диалога с чат-ботом: это учебный артефакт, согласованный по структуре, терминологии и уровню сложно-

сти. Gupta et al. [7] показали, что при математическом тьюторинге LLM дают правильный финальный ответ в 85,5% случаев, но лишь 56,6% диалогов оказываются полностью корректными педагогически. Это расхождение указывает на необходимость внешнего контроля структуры и содержания.

Такой контроль обеспечивают методы управляемой генерации [15]: конспект строится по фиксированной структуре обязательных блоков (определения, примеры, разбор типичных ошибок, визуальные пояснения), а LLM адаптирует содержание каждого блока под профиль ученика. Фиксированная структура выполняет роль внешнего каркаса: она не ограничивает содержание, но гарантирует, что конспект покрывает все необходимые элементы и сохраняет последовательность изложения. Проблема галлюцинаций [9] в учебном контексте особенно опасна: некорректное определение или ошибка в формуле может закрепить ложное знание. В отличие от диалогового сценария, где ученик может переспросить и получить уточнение, конспект воспринимается как авторитетный источник, поэтому требования к фактической точности здесь выше. Это обосновывает привязку генерации к внешнему корпусу проверенных материалов.

RAG [13] решает эту проблему: модель обращается к внешнему корпусу, что повышает точность и позволяет обновлять базу знаний без переобучения. Обзор [14] фиксирует специфику образовательного RAG: в отличие от общего случая, здесь требуется курируемый корпус (учебники, методические материалы, задания с разборами), а извлечённые фрагменты должны соответствовать уровню ученика. Henkel et al. [8] продемонстрировали улучшение качества ответов на математические вопросы при использовании RAG с опорой на учебный источник. В разрабатываемой системе корпусом служат материалы ФИПИ [36], что обеспечивает соответствие школьной программе и требованиям экзамена.

Третий компонент пайплайна связан с визуализацией. В математическом обучении графики, чертежи и блок-схемы выполняют содержательную функцию: связывают формулу с визуальным представлением, иллюстрируют пространственные отношения. Для ряда тем ЕГЭ визуальное представление является не дополнением к тексту, а основным способом понимания: задачи на графики функций, на взаимное расположение фигур, на анализ производной по графику требуют умения читать и строить диаграммы. Конспект, лишённый визуальной составляющей, для таких тем оказывается неполноценным. Мета-анализ [19] подтверждает положительный эффект визуализаций, но указывает, что он зависит от типа визуализации, её связи с текстом и когнитивной нагрузки.

Модели генерации изображений по тексту (text-to-image) плохо подходят для этой задачи: они нарушают точные геометрические отношения и некорректно синтезируют под-

писи и формулы [3]. Wei et al. [26] формулируют задачу генерации диаграмм по тексту как отдельную и подчёркивают, что результат должен быть редактируемым и проверяемым. Для геометрии Wang et al. [25] показали, что корректные пространственные отношения требуют формальных ограничений, поскольку свободная генерация систематически их нарушает. Эти результаты обосновывают архитектуру, принятую в данной работе: языковая модель формирует JSON-спецификацию визуального элемента, методы ограниченной генерации (constrained generation) [27] гарантируют валидность выходного JSON, а специализированный движок рендеринга строит изображение в формате масштабируемой векторной графики (Scalable Vector Graphics, SVG), обеспечивая геометрическую корректность и возможность адаптации под уровень ученика.

Каждый из трёх компонентов решает свою задачу, но ни один по отдельности не закрывает потребность в персонализированном конспекте с визуальными материалами. Управляемая генерация обеспечивает структуру, но не гарантирует фактическую точность. RAG повышает точность, но не создаёт визуальных материалов. Структурная генерация диаграмм обеспечивает корректные иллюстрации, но без привязки к профилю ученика они остаются универсальными. Их интеграция в единый пайплайн, связанный с моделью компетенций ученика, составляет архитектурную основу разрабатываемой системы.

1.3 Существующие решения и методы оценки качества

Сравнительный анализ аналогов решает две задачи: показать, что рынок не пуст и отдельные компоненты предлагаемой системы уже реализованы в тех или иных продуктах, и одновременно продемонстрировать, что ни одно из существующих решений не объединяет персонализацию на основе аналитики ученика, автоматическую генерацию конспектов и программное создание точных математических визуализаций.

В российском сегменте подготовки к ЕГЭ по математике доминируют онлайн-школы с видеоуроками и практическими заданиями: Фоксфорд [37], Умскул [33], Школково [38]. Эти платформы предлагают структурированные курсы и обратную связь от преподавателей, однако не реализуют автоматическую генерацию персонализированного учебного контента. Портал РЕШУ ЕГЭ [32] предоставляет обширный банк заданий с возможностью тематической фильтрации, но без адаптации под профиль ученика. Платформы с элементами адаптивности, такие как Учи.ру [34] и Яндекс Учебник [39], используют машинное обучение для подбора заданий и предлагают AI-тьюторов по математике, однако их функциональность сосредоточена на диалоговом формате, а не на генерации конспектов с визуальными

материалами.

Среди международных продуктов наиболее близким по идеологии является Khanmigo [10] от Khan Academy, который акцентирует педагогическую рамку (не выдавать готовый ответ, направлять к пониманию). Специализированные математические инструменты, такие как ALEKS [17] и MATHia [4], реализуют сильную адаптивную аналитику на основе теории пространств знаний и когнитивных моделей, но работают в формате пошаговых заданий, а не генерации учебных материалов.

В Таблице 1.1 представлено сравнение репрезентативных платформ по четырём признакам, непосредственно связанным с архитектурой разрабатываемой системы.

Таблица 1.1: Сравнение существующих решений по ключевым функциональным признакам

Платформа	Персонализация контента	Генерация конспектов	Авто- визуализации	Аналитика ученика
Фоксфорд	—	—	—	базовая
Умскул	—	—	—	базовая
РЕШУ ЕГЭ	—	—	—	базовая
Учи.ру	+	—	—	+
Яндекс Учебник	+	частично	—	+
Khanmigo	+	частично	—	+
ALEKS	+	—	—	++
MATHia	+	—	—	++
Данная работа	+	+	+	+

Как видно из таблицы, платформы с сильной аналитикой (ALEKS, MATHia) не генерируют учебный контент автоматически, а продукты с элементами генерации (Khanmigo, Яндекс Учебник) работают в диалоговом формате и не создают структурированных визуализаций. Ни одно из рассмотренных решений не реализует полный цикл создания персонализированных конспектов, что определяет нишу данной работы.

Это расхождение не случайно: оно отражает разницу в архитектурных приоритетах. Платформы с глубокой аналитикой исторически строились вокруг адаптивного тестирования и подбора заданий, а не генерации объяснительного контента. Продукты с генеративными возможностями, напротив, появились с распространением LLM и пока сосредоточены на диалоговом взаимодействии, где визуальный контент не является ключевым элементом. Предлагаемая система занимает промежуточную позицию: она использует аналитику ученика для управления генерацией, а генерацию выстраивает как создание законченного учебного артефакта с текстом и визуальными материалами.

2 Архитектура системы и подход к реализации

Обзор литературы показал, что ни одна из существующих платформ не реализует полный цикл от аналитики ученика к персонализированному конспекту с математическими визуализациями. В настоящем разделе описывается, как этот цикл устроен в разрабатываемой системе: какова её общая архитектура, как устроены её модули и какие инженерные решения положены в их основу. Ограничения текущей реализации и перспективные направления развития рассматриваются отдельно в подразделе [3.12](#).

2.1 Общая архитектура системы

Разрабатываемый модуль встраивается в платформу «Прорыв» [\[30\]](#) в качестве компонента, отвечающего за генерацию учебных материалов. На входе модуль получает вектор компетенций ученика, на выходе — структурированный конспект с текстом и визуальными иллюстрациями, адаптированными под его уровень.

Вектор компетенций формируется по истории взаимодействий ученика с платформой: из решённых задач, допущенных ошибок и затраченного времени строится оценка уровня освоения по каждой теме. Тематическое пространство опирается на кодификатор ФИПИ [\[35\]](#), однако не совпадает с ним в точности: темы кодификатора служат основой, которая при необходимости детализируется или перегруппируется с учётом характерных паттернов ошибок учеников. Такая привязка принципиальна: она соотносит профиль ученика с реальной структурой экзамена. Вектор компетенций выполняет роль управляющего сигнала на всех последующих этапах генерации: определяет, какие темы включить в конспект, какой уровень детализации выбрать и где добавить дополнительные визуальные пояснения.

Генерация конспекта организована как двухэтапный процесс. На первом этапе языковая модель формирует текстовое содержание по фиксированной структуре обязательных блоков, адаптируя каждый блок под профиль ученика. На втором этапе отдельный модуль получает текстовые описания нужных иллюстраций и строит по ним математические диаграммы. Оба этапа описаны в следующих подразделах.

2.2 Генерация текстового содержания конспекта

Центральный элемент первого этапа — управляемая генерация текста (controllable generation) [\[15\]](#). Конспект строится по фиксированной структуре, включающей несколько обязательных блоков: определения и формулировки, разбор типичных ошибок, примеры за-

дач с пошаговым решением и визуальные пояснения. Фиксированная структура гарантирует, что конспект покрывает все необходимые элементы независимо от темы и уровня ученика. Языковая модель адаптирует содержание каждого блока под вектор компетенций: для ученика с низким освоением темы включаются промежуточные шаги рассуждения и больше примеров, для более подготовленного ученика изложение компактнее.

Архитектура модуля намеренно построена без привязки к конкретной языковой модели: интерфейс взаимодействия с моделью унифицирован, что позволяет подменять её без изменения остальных компонентов системы.

Фактическая точность содержания обеспечивается за счёт генерации с извлечением информации (retrieval-augmented generation, RAG) [13]. Корпус для поиска формируется из материалов открытого банка заданий ФИПИ [36], что привязывает содержание конспекта к требованиям экзамена. На этапе генерации модель обращается к извлечённым фрагментам, что снижает риск фактических ошибок: некорректное определение или ошибочная формула в учебном материале воспринимается учеником как авторитетное утверждение и может закрепить ложное знание [9].

Модуль текстового этапа формирует все обязательные блоки конспекта в соответствии с фиксированной структурой и определяет места для визуальных элементов, передаваемых на следующий этап.

2.3 Генерация визуализаций

Второй этап отвечает за автоматическое создание математических иллюстраций. Прямое использование моделей генерации изображений по тексту (text-to-image) для этой задачи неприемлемо: такие модели систематически нарушают точные геометрические отношения и некорректно воспроизводят подписи и математические формулы [3, 25]. В данной работе принят альтернативный подход: языковая модель формирует не изображение, а структурированную спецификацию визуального элемента, по которой специализированный движок строит итоговое изображение.

Процесс состоит из трёх шагов. На первом шаге языковая модель, опираясь на текстовое описание нужной иллюстрации, генерирует JSON-спецификацию визуального элемента. Формат спецификации разработан в рамках данной работы и содержит семантически значимые поля: тип диаграммы, перечень объектов с их параметрами, подписи и пространственные ограничения. На втором шаге корректность выходного JSON обеспечивается библиотекой Instructor, которая валидирует результат по Pydantic-схеме и при необходимости автомати-

чески повторяет запрос. На третьем шаге специализированный движок разбирает спецификацию и строит изображение в формате масштабируемой векторной графики (Scalable Vector Graphics, SVG) [24]. SVG выбран потому, что он обеспечивает геометрическую корректность без потерь при масштабировании, поддаётся программной проверке и может быть встроен непосредственно в веб-интерфейс платформы.

Как и в модуле генерации текста, архитектура не зависит от конкретной языковой модели: замена модели не требует изменений в схеме JSON или логике рендеринга.

Реализованный движок поддерживает три типа математических иллюстраций.

Графики функций. Движок строит графики элементарных и составных функций, встречающихся в задачах ЕГЭ: степенных, показательных, логарифмических, тригонометрических. Все значения функции вычисляются программно по аналитическому выражению, что исключает погрешности ручного построения. Поддерживаются подписи осей, отметки характерных точек (нули, экстремумы, точки пересечения).

Геометрические чертежи. Для задач планиметрии и стереометрии движок строит фигуры по заданным параметрам с корректными пространственными соотношениями. Корректность обеспечивается на уровне спецификации: JSON-формат содержит явные ограничения (параллельность, перпендикулярность, равенство отрезков), которые движок соблюдает при построении. Этот подход согласуется с выводами Wang et al. [25], показавших, что корректные пространственные отношения требуют формальных ограничений, поскольку свободная генерация систематически их нарушает. Также есть возможность создавать дополнительные построения, указывая только их смысл, например «медиана из угла В», а координаты вычисляются программно.

Блок-схемы. Для алгоритмических задач и объяснения многошаговых методов решения движок строит блок-схемы с соединительными стрелками и условными ветвлениями. Есть возможность добавлять подписи к стрелкам. Также реализована возможность отрисовывать формулы и математические символы в формате LaTeX.

Этап генерации визуализаций реализован для всех трёх типов иллюстраций. Расширение набора поддерживаемых геометрических примитивов и работа над качеством подписей при сложных конфигурациях выполнены в виде последовательных итераций улучшений и подробно описаны в подразделе 3.10.

2.4 Сценарии использования и контур развёртывания

Разработанный модуль предназначен для использования в составе сервиса «Прорыв» в качестве компонента генерации учебных материалов. Целевой пользователь — школьник, самостоятельно готовящийся к ЕГЭ по математике.

Связь между аналитическим профилем и генератором осуществляется через вектор компетенций: каждое обращение к модулю сопровождается актуальным состоянием профиля, что позволяет корректировать содержание конспекта при изменении уровня ученика. Если ученик систематически допускает ошибки при исследовании функции на монотонность, сгенерированный конспект уделяет этому пункту больше внимания и включает дополнительный иллюстративный график.

Модуль реализован как самостоятельный компонент с унифицированным интерфейсом обращения к языковой модели и фиксированным форматом входных и выходных артефактов. Такой контур упрощает воспроизводимость экспериментов: прогон системы не зависит от производственной инфраструктуры платформы и допускает автономный запуск на отдельной рабочей машине. Сопряжение модуля с действующим сервисом относится к эксплуатационным задачам и рассматривается среди направлений развития (подраздел 3.12).

2.5 Оценка качества

Оценка качества прототипа в рамках настоящей работы построена на двух уровнях: техническая валидация процесса генерации и качественный анализ полученных конспектов и иллюстраций. Принятый объём оценки соответствует масштабу прототипа. На прототипном корпусе содержательные выводы о работоспособности контура и о направлениях его улучшения получаются именно из автоматических метрик и предметного разбора выходов, тогда как полноценная проверка образовательного эффекта на реальных учениках имеет смысл уже после первичной отладки и интеграции системы в действующий сервис, и в прототипной фазе настоящей работы не проводилась. Обоснование такого решения и возможные методические подходы к проверке на пользователях обсуждаются среди направлений развития (подраздел 3.12).

Техническая валидация. Технические свойства конвейера измерены автоматическими метриками на корпусе из четырёх учебных конспектов (вероятность, уравнения и неравенства, производная и её приложения, прикладная математика) и восьми отдельных задач планиметрии и стереометрии. По конспектной ветке проведена серия из трёх независимых прогонов с фиксацией числа запросов, объёма входных и выходных токенов, средней

задержки генерации и числа запланированных визуалов. Результаты сведены в табл. 3.3 в подразделе 3.7. Дополнительно выполнены температурная абляция планировщика (табл. 3.5) и сопоставление двух языковых моделей разной размерности на одинаковом наборе входов (табл. 3.11). Геометрическая ветка получила симметричное журналирование на уровне отдельных шагов: на корпусе из восьми задач проведена серия повторных прогонов с агрегированием времени, числа токенов, доли планов, потребовавших исправления (fix-rate), и доли прогонов с визуализированным ответом. Результаты сведены в табл. 3.10. Эффект включения пост-валидатора визуализации ответа в режим замкнутого цикла исправления зафиксирован отдельным экспериментом (табл. 3.9). Корректность вычислений агрегаторов и пост-валидатора покрыта изолированными тестами модулей `pipeline/_smoke_geometry_variance` и `pipeline/_test_answer_validator.py`, не требующими обращения к языковой модели.

Качественный анализ конспектов и иллюстраций. Сгенерированные конспекты и геометрические чертежи проанализированы авторами работы на том же корпусе. Этот анализ выявил характерные классы технических сбоев (ошибки разбора JSON, нарушения ссылочной целостности в плане, коллизии подписей, обрезка точек границами области, ошибки порядка обхода вершин в трёхмерных гранях, пропуск визуального представления ответа) и характерные педагогические дефекты, систематически описанные в подразделе 3.11. Разбор стал содержательной базой для тринадцати итераций улучшений (подраздел 3.10): каждая итерация привязана к конкретному классу наблюдавшихся ошибок и переносит часть рассуждения с языковой модели на детерминированные алгоритмы. На прототипном масштабе именно такой предметный разбор обеспечивает содержательную валидацию инженерных решений и позволяет поставить осмысленные вопросы для последующего экспертного и полевого исследований.

Сочетание двух описанных уровней — технического и качественного — достаточно для того, чтобы зафиксировать работоспособность контура и обоснованность принятых инженерных решений на прототипном корпусе. Более развёрнутые формы валидации (формальное сравнение с базовой линией text-to-image, экспертная разметка иллюстраций учителями математики, контролируемое тестирование на учениках) требуют отдельных протоколов, экспертного аппарата и доступа к производственному окружению. Их обоснование, возможные методические подходы и место в плане работ обсуждаются среди направлений развития системы (подраздел 3.12).

3 Генерация визуализаций: реализация и эксперименты

3.1 Постановка задачи

Введение и раздел об архитектуре системы (подраздел 2) обосновали отказ от прямого порождения растровых иллюстраций моделями текста к изображению (text-to-image) и переход к двухшаговой схеме, где языковая модель формирует структурированную спецификацию сцены, а специализированный движок строит файл векторной графики. В настоящей главе этот выбор переводится в плоскость инженерных решений: описывается архитектура реализованного прототипа, формат внутреннего представления, отдельные модули и эксперименты, которые показали работоспособность контура от текста конспекта до итогового SVG.

Прототип реализован на материалах ЕГЭ по математике. Этот выбор обусловлен сочетанием педагогической и инженерной мотивации. Со стороны педагогики, математика — предмет с наиболее формализованным языком и хорошо развитой типологией визуальных представлений: графики функций, числовые прямые, геометрические чертежи, диаграммы знаков производной и деревья исходов составляют устойчивый репертуар иллюстраций, известный по учебникам и методическим пособиям. Со стороны инженерии, наличие формализованного решателя геометрических ограничений и описуемого языка плоских и пространственных построений упрощает проверку гипотезы о двухшаговой генерации с детерминированной верификацией: согласованность построенной сцены с условием задачи можно для большинства типичных дефектов проверять алгоритмически, не прибегая к экспертной оценке. При этом архитектура контура — двухшаговая декомпозиция, промежуточный JSON-формат описания сцены и реестр стратегий конвертации — спроектирована с заделом на расширение на другие предметные области профильного ЕГЭ (физика, химия, биология, информатика, обществознание). Конкретные направления такого обобщения и оценка инженерных затрат на него вынесены в подраздел 3.12.

Глава решает три прикладные задачи. Первая задача состоит в том, чтобы зафиксировать архитектуру и отразить её в виде программных модулей. Вторая задача состоит в накоплении эмпирических наблюдений о поведении контура на материалах ЕГЭ по математике: где он работает корректно, где обнаруживаются ограничения, какие из них можно устранить без смены модели, а какие требуют дополнительной инженерии. Третья задача состоит в систематизации найденных проблем и описании итераций улучшения: каждая итерация привязана к конкретному классу ошибок, наблюдавшихся на предыдущем шаге.

Эксперименты в данной главе сопровождают реализацию системы и направлены на проверку инженерной связности конвейера, измерение его поведения на учебном корпусе и количественную оценку ключевых характеристик. Измеряются следующие свойства системы. Во-первых, устойчивость состава визуальных элементов, отбираемых языковой моделью по тексту конспекта, при независимых повторных запусках на одном и том же входе. Во-вторых, зависимость этой устойчивости от параметра температуры языковой модели. В-третьих, доля прогонов, в которых автоматическая проверка структурированного выхода модели обнаружила несогласованность и инициировала запрос на исправление. В-четвёртых, эффективность автоматического цикла, исправляющего педагогические дефекты чертежа — ситуации, когда искомая величина показана только текстовой подписью, а не геометрическим объектом. В-пятых, сопоставление двух языковых моделей разной размерности на одном и том же корпусе. Полученные оценки приводятся в виде сводных таблиц с явным указанием состава корпуса и числа независимых прогонов.

Изложение организовано в логике от общего к частному и далее к итерациям. Сначала описывается архитектура верхнего уровня и поток данных. Затем последовательно раскрываются три ключевые ступени конвейера: планировщик, конвертер и движок рендеринга. После этого приводится первая интеграционная проверка на четырёх учебных конспектах. Дальнейшие подразделы рассматривают расширение в сторону пошаговой геометрии и стереометрии, набор итеративных улучшений и анализ предельных случаев.

3.2 Архитектура программного контура

Прототип реализован как набор взаимосвязанных модулей в каталоге `pipeline` проекта. Поток данных от исходного учебного материала к иллюстрации проходит через четыре ступени: разбиение текста конспекта на смысловые секции, планирование визуальных элементов на уровне отдельной секции, конвертацию плана в описание сцены и рендеринг сцены в файл SVG. Такое разбиение отвечает классической логике многоступенчатых обработчиков естественного языка, в которых каждая ступень имеет узкую ответственность и фиксированный контракт по входу и выходу. Подобная декомпозиция также облегчает диагностику: ошибка отображения может находиться либо в извлечении параметров языковой моделью, либо в стратегии конвертации, либо в самом строителе графики, и эти источники локализуются независимо.

Входные данные. Конспект подаётся в виде Markdown-файла. Внутри он размечен заголовками, краткими определениями, формулами в синтаксисе LaTeX и примерами решений.

Принятая форма исходных материалов позволяет работать с ними как с обычным текстом естественного языка, не требуя предварительной структурной разметки в специализированных форматах.

Разбиение на секции. Первая ступень разбивает документ на пронумерованные смысловые единицы по структуре заголовков. Слишком короткие фрагменты (например, отдельные оглавления и фрагменты с одним выражением) отбрасываются как неподходящие для постановки визуального элемента. Эта эвристика снижает количество вырожденных запросов к языковой модели и удерживает внимание планировщика на содержательных секциях.

Планировщик визуалов. Вторая ступень принимает решение, требуется ли для каждой секции иллюстрация, и если да, какого она должна быть типа. В прототипе используется планировщик второй версии, который работает в две стадии. На первой стадии модель смотрит на полный список секций и помечает кандидатов на визуализацию. На второй стадии для каждого кандидата извлекаются параметры, специфичные для типа: для графика функции это выражение и интервал, для диаграммы знаков производной это критические точки, для дерева исходов это корневая постановка и список ветвей. Двухпроходная схема описана подробнее в подразделе [3.4](#).

Конвертер. Третья ступень соединяет педагогически богатую таксономию планировщика с более узким набором сцен, на котором работает движок графики. Эта операция нетривиальна: одному типу плана может соответствовать несколько разных типов сцен, а одному типу сцены могут соответствовать разные типы плана. Конвертер реализует регистрацию стратегий по типу плана: каждая стратегия знает, как из полей плана сформировать список объектов сцены и аннотаций. Подробное описание дано в подразделе [3.5](#).

Движок рендеринга. Четвёртая ступень принимает описание сцены и формирует SVG. Основной путь предполагает обращение к полноценному сервису SVG-генератора, реализованному в отдельном подкаталоге `svg-generator`: в этом сервисе на схемах Pydantic построен решатель геометрических ограничений — компонент, который по семантически заданным связям между объектами (например, «отрезок BH перпендикулярен прямой AD ») сам вычисляет координаты вспомогательных точек, — а также набор примитивов, охватывающий координатные оси, кривые, окружности, многоугольники и пр. На случай, если зависимости сервиса по каким-то причинам недоступны (например, при несовместимости двоичных модулей в среде разработки), в `run_pipeline.py` предусмотрен встроенный облегчённый ренде-

рер, поддерживающий минимально необходимые сцены. Каждая ступень фиксирует журнал замечаний (warnings), которые в дальнейшем используются для оценки качества контура.

Выходные артефакты. Для каждого конспекта прогон создаёт каталог с тремя слоями данных: `plan.json` с полным выходом планировщика, набор файлов `scene_*.json` со спецификациями сцен и набор файлов `visual_*.svg` с готовыми иллюстрациями. Дополнительно собирается общий `results.json` и страница `preview.html` с визуальной сводкой по всем конспектам. Данный набор артефактов важен для воспроизводимости экспериментов: повторный рендеринг можно проводить из сохранённых планов без новых обращений к языковой модели, что снижает разброс результатов и стоимость прогонов.

Распределение зон ответственности. Прототип реализован в команде из двух авторов выпускной квалификационной работы. В сферу ответственности А. С. Низова входили двухпроходный планировщик визуалов конспекта и педагогическая таксономия его типов, а также отдельная ветка пошаговой геометрии и стереометрии с двухступенчатым обращением к языковой модели, циклом исправлений по ссылочной целостности и автоматическим определением видимости рёбер. На геометрической и конспектной частях корпуса им проведены количественные эксперименты: серия повторных прогонов с агрегированием fix-rate валидатора плана и доли прогонов с визуализированным ответом, эксперимент с переводом пост-валидатора визуализации ответа в режим замкнутого цикла исправлений, температурная абляция планировщика и сопоставление двух языковых моделей разной размерности на одинаковом наборе входов (подразделы [3.4](#), [3.8](#), [3.9](#), [3.7](#), [3.11](#)). За вторую половину конвейера отвечал Д. А. Ковыркин: конвертер плана в описание сцены, движок рендеринга со схемами Pydantic и решателем геометрических ограничений, а также широкий набор инженерных улучшений пошаговой генерации — тринадцать итераций над визуальным представлением и инфраструктурой конвейера (валидация плана с повторным запросом на исправление, ограничение пересечения прямых, антиколлизия подписей, авторасчёт области отрисовки на плоскости, санитизация JSON, инструментирование журнала прогона, единая точка входа в обе ветки конвейера) (подразделы [3.5](#), [3.6](#), [3.10](#)). Промежуточный JSON-формат описания сцены и схема журналирования прогона выполнены и сопровождаются совместно: фиксированный контракт между двумя половинами конвейера позволяет развивать каждую ветку независимо, не нарушая воспроизводимости прогонов.

Объём программной реализации и публикация исходного кода. Совокупный объём исходного кода прототипа составляет 17 560 строк в 66 файлах языка Python общим объёмом около 736 КБ. Распределение объёма по основным модулям конвейера приведено в табл. 3.1: наибольшая доля приходится на ветку пошаговой геометрии и стереометрии (геометрический планировщик, рендерер с автоматическим определением видимости рёбер, демонстрационные сценарии) и на инфраструктуру экспериментов (агрегаторы метрик, прогон серий, выгрузка пошаговых SVG, тесты модулей). Полные исходные тексты прототипа, LaTeX-исходники настоящей работы и сопровождающий графический материал размещены в открытом репозитории `kovyrkov_nizov_vkr`¹. Это позволяет читателю обратиться к коду конкретного модуля при чтении соответствующих подразделов главы и воспроизвести описанные ниже эксперименты на тех же входных данных.

Таблица 3.1: Объём программной реализации прототипа в разрезе модулей конвейера.

Модуль	Файлов	Строк кода	Размер, КБ
Конспектная ветка (планировщик и оркестратор)	5	2 204	87,1
Геометрическая ветка (пошаговая геометрия и стереометрия)	11	7 925	370,1
Экспериментальная обвязка и метрики	14	3 688	140,0
Эксперименты с наивными бэйзлайнами	1	181	6,6
SVG-генератор (рендерер и решатель ограничений)	35	3 562	132,1
Итого	66	17 560	735,9

Описанный поток данных схематически приведён на рис. 3.1. На рисунке отображены границы ответственности каждой ступени и формы промежуточных артефактов, этим он отличается от более общей архитектурной схемы из подраздела 2 и предназначен для подробного обсуждения именно в данной главе.

3.3 Промежуточный формат описания сцены

Ключевая роль в архитектуре отведена промежуточному JSON-формату описания сцены, который называется Scene-форматом. Он играет роль контракта между языковой моделью и движком графики, позволяя независимо развивать обе стороны контура. На входе у движка нет необработанного текста модели: только нормализованные объекты, прошедшие валидацию по схеме.

Сцена в прототипе характеризуется одним из трёх возможных типов: `function_plot`, `geometry` или `diagram`. Тип определяет ветвь рендеринга и список допустимых объектов.

¹https://github.com/GitDenK/kovyrkov_nizov_vkr

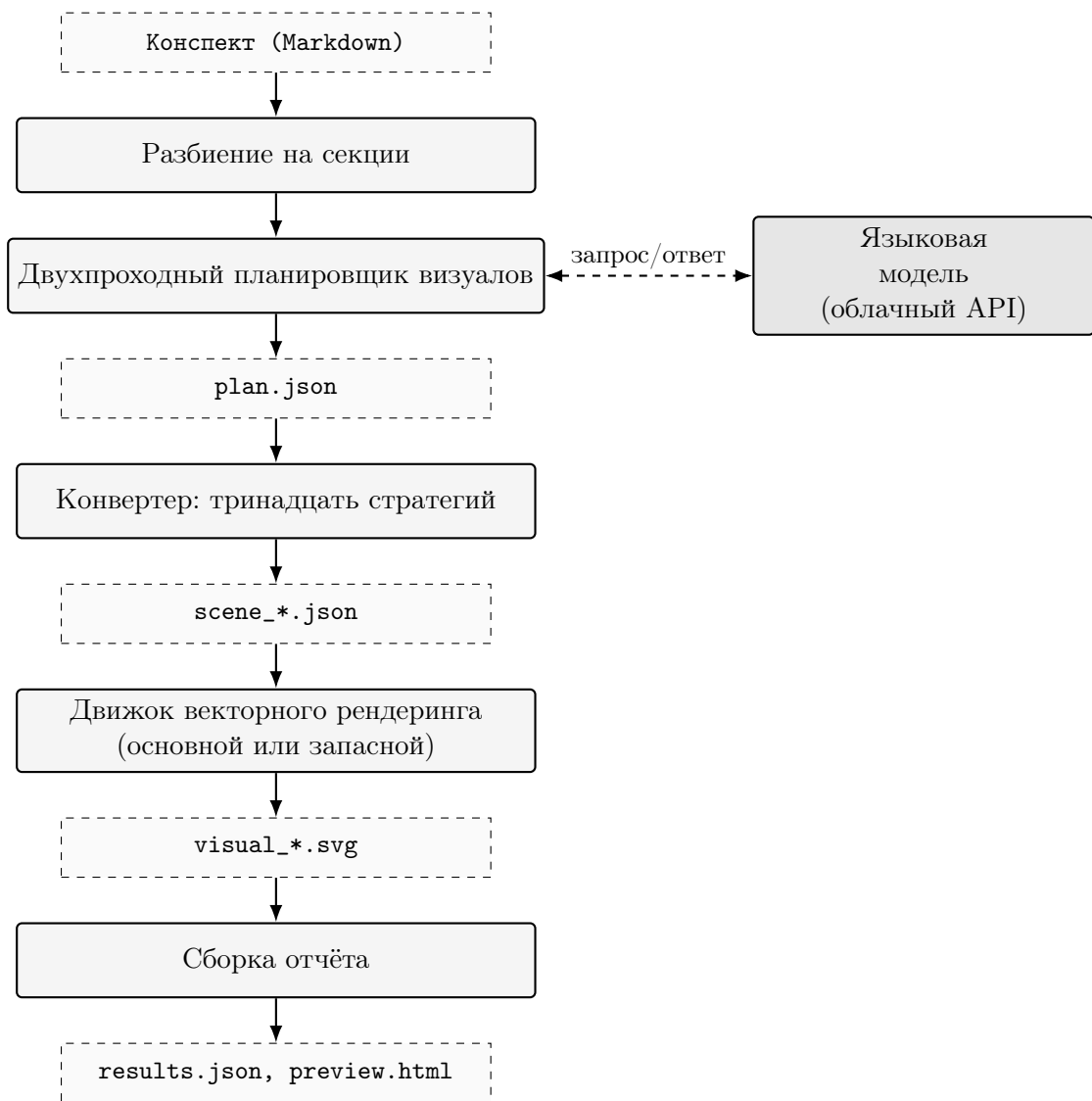


Рис. 3.1: Поток данных от учебного конспекта к итоговому файлу SVG в реализованном прототипе. Сплошные стрелки соответствуют детерминированным шагам конвейера, штриховая стрелка обозначает обращение к внешней языковой модели через сетевой интерфейс. Прямоугольники с штриховой границей соответствуют файлам промежуточных артефактов, сохраняемым на диске для обеспечения воспроизводимости прогона.

Сцена типа `function_plot` предназначена для координатных построений и содержит координатные оси, кривые с аналитическими выражениями, точки и аннотации. Сцена типа `geometry` предназначена для планиметрических и стереометрических чертежей и содержит точки, отрезки, многоугольники, окружности, а также набор геометрических ограничений (constraints), которые движок разрешает до отрисовки. Сцена типа `diagram` предназначена для схематических иллюстраций и содержит блоки, стрелки и формульные элементы.

Несмотря на разную природу трёх типов сцен, их объединяет принцип семантической, а не позиционной спецификации. Это означает, что языковой модели поручается описать смысл иллюстрации, а не её пиксельные координаты. Например, в сцене типа `geometry` модель указывает, что отрезок BH перпендикулярен прямой AD и опущен из вершины B , но не вычисляет координат точки H . Координаты основания перпендикуляра вычисляются решателем по правилу, определённого типу ограничения `perpendicular`. Такое разделение ответственности повышает геометрическую корректность результата и снижает требования к арифметической аккуратности модели.

Описанная семантика опирается на представление о структурированной генерации, где выход модели проверяется внешней схемой. В сервисе SVG-генератора эта схема описана в виде классов Pydantic, а ограниченная генерация (constrained generation) [27] обеспечивает соответствие выхода схеме на уровне декодирования. В исследовательской ветке прототипа, работающей через простой HTTP-клиент облачного API, проверка проводится постфактум: после получения ответа JSON разбирается, при необходимости очищается от типичных синтаксических ошибок (см. подраздел 3.10), затем проверяется на ссылочную целостность точек, рёбер и ограничений. Расхождение в способе обеспечения валидности выхода между двумя ветками является осознанным архитектурным компромиссом, разделяющим продуктовый и исследовательский режимы работы.

Промежуточный формат не привязан к конкретной языковой модели и к конкретному рендереру. Это позволяет менять модель, не изменяя движок, и наоборот, расширять движок (например, добавлять новые типы примитивов), не вмешиваясь в логику планирования. Универсальность формата подтверждается тем, что один и тот же `function_plot` обслуживает графики функций, диаграммы знаков производной, числовую прямую и точки на координатной плоскости, тогда как один и тот же `diagram` обслуживает таблицы, формульные карточки, шаговые схемы решения и деревья исходов. Подробное соответствие приведено в подразделе 3.5.

3.4 Двухпроходный планировщик визуалов

Планировщик второй версии решает задачу извлечения структурированных описаний визуальных элементов из текста конспекта. Эта задача отличается от типичных постановок суммаризации или классификации: модель должна не только понять, о чём говорится в секции, но и установить, поможет ли иллюстрация ученику в понимании, и если да, какие параметры она должна включать.

Архитектурно планировщик реализован как двухпроходный модуль. На первом проходе модель получает обзор конспекта в виде нумерованного списка заголовков и кратких описаний, после чего возвращает решения по каждой секции в виде пары: нужен ли визуал и какого он типа. Доступная таксономия типов в текущей реализации насчитывает тринадцать категорий, разделённых по педагогическому смыслу: `function_graph`, `tangent_line_graph`, `derivative_sign_chart`, `number_line`, `coordinate_plane`, `venn_diagram`, `step_by_step_diagram`, `flowchart`, `formula_card`, `outcome_tree`, `comparison_chart`, `table`, `annotated_example`. Такая дробная типология обусловлена тем, что разные педагогические сценарии требуют принципиально разных подсказок к рендереру: например, дерево исходов и сравнительная таблица обе попадают в общий тип сцены `diagram`, но строятся из разного набора объектов, поэтому планировщик различает их на этапе планирования, а не на этапе рендеринга.

На втором проходе для каждой отобранной секции выполняется отдельный запрос к языковой модели, в котором извлекаются параметры визуала. Структура запроса зависит от типа. Для `function_graph` извлекаются аналитическое выражение функции, область определения переменной и характерные точки. Для `derivative_sign_chart` извлекаются критические точки и интервалы знакопостоянства. Для `outcome_tree` извлекается формулировка корневой задачи и список вариантов исхода с краткими подписями. Если ответ модели не удаётся разобрать как валидный JSON либо в нём отсутствуют ключевые параметры, секция помечается как пропущенная: общий контур переходит к следующей.

Системный промпт первого прохода. Принципы работы первого прохода фиксируются в системном промпте, который сопровождает каждый запрос к модели. Помимо постановки задачи и перечня тринадцати допустимых типов промпт содержит три правила отбора, формализующие педагогическую оценку секции:

Правило 1. Визуал нужен для 40–70 % секций — не каждой, но и не слишком мало.

Правило 2. Визуал обязательно нужен для ключевых формул, примеров ре-

шений с вычислениями, описаний типов и классификаций, алгоритмов решения, описаний функций и графиков.

Правило 3. Визуал не нужен для мотивационных блоков и общих советов, для очень коротких секций (только заголовки без содержания) и для секций, в которых уже размещена корректно оформленная markdown-таблица.

Эта формулировка задаёт правовое поле, в котором работает планировщик, и одновременно очерчивает зону неопределённости на стыке правил 2 и 3 (содержательная иллюстрация эффекта приведена в подразделе 3.7). В качестве выхода первого прохода модель возвращает JSON-массив с одной записью на секцию: `section_id`, флаг `need_visual`, выбранный `visual_type` и короткое словесное обоснование `reason`. Поле `reason` не используется конвейером функционально, но сохраняется в журнал прогона и оказывается полезным при последующем анализе пограничных случаев.

Системный промпт второго прохода. Для каждой отобранной секции запускается отдельный запрос со своим системным промптом. Этот промпт уже знает тип визуала, выбранный на первом проходе, и формулирует задачу извлечения параметров. Ключевое требование промпта — конкретизация: в полях параметров должны фигурировать настоящие числа, формулы и значения, взятые из текста секции, а не абстрактные плейсхолдеры. Соответствующая инструкция в промпте звучит так:

В `params` используй конкретные числа, формулы и значения из текста секции. Не пиши абстрактные `x1`, `x2`, `f(x)` — извлекай реальные данные из текста.

Чтобы модель надёжно следовала этому правилу, в промпт включён набор few-shot примеров (по одному на каждую крупную семью типов). Например, для визуала типа `derivative_sign_chart` отвечающего за разметку интервалов знакопостоянства производной, в промпт подставлен следующий образец ответа:

```
{"visual_type": "derivative_sign_chart",
  "params": {
    "function": "x^2 - 4x + 5",
    "critical_points": [2],
    "intervals": [
      {"range": "(-inf, 2)", "sign": "-", "behavior": "убывает"},
      {"range": "(2, +inf)", "sign": "+", "behavior": "возрастает"}
    ]
  }
}
```

```

],
  "extrema": [{"x": 2, "type": "минимум"}]
}
}

```

Образец демонстрирует не только формальную структуру JSON, но и желаемый уровень конкретики: указана конкретная квадратичная функция, перечислены критические точки, знаки производной на интервалах и точка минимума. На вход второго прохода поступает связка из системного промпта с примерами и одной пользовательской реплики с полным текстом секции, по которому модель должна сформировать аналогичный JSON. В случае невалидного выхода (синтаксическая ошибка JSON, отсутствие обязательных полей) секция помечается как пропущенная без отдельных попыток исправления.

Разделение задачи на два прохода имеет два мотива. Во-первых, оно отделяет грубое решение «нужен ли визуал и какого типа» от более тонкого извлечения параметров. На больших конспектах из 15–20 секций модель в одном проходе склонна забывать про начальные секции к моменту обработки концовки документа. Двухпроходная схема снимает эту проблему: на первом проходе обрабатывается весь документ, на втором каждый кандидат рассматривается отдельно. Во-вторых, она снижает риск дорогих обращений к модели на секциях, в которых визуал не нужен: на втором проходе обрабатываются только отобранные кандидаты.

Стоимость двухпроходной схемы тоже стоит обсудить. Первый проход выполняется в одном запросе на весь документ, второй проход требует отдельного запроса на каждый отобранный кандидат. Для типового конспекта из 14–20 секций, в котором отбирается 7–13 кандидатов, общее число запросов к модели за один прогон планировщика составляет от 8 до 14. Этот порядок согласуется с журналами прогонов, описанных в подразделе 3.7. Чтобы оценить разброс измерений по серии независимых запусков, конспектная ветка конвейера была прогнана трижды на одном и том же корпусе из четырёх конспектов. Совокупное время на этап планирования по корпусу составило в среднем 173 секунды на один прогон (диапазон от 156 до 191 секунды), суммарный объём контекста — 81 707 входных и 14 504 выходных токенов в среднем. Стандартное отклонение по числу запланированных визуалов составило 5 единиц при среднем значении 38, что в относительном выражении составляет около 13 %. Эти оценки получены на трёх независимых прогонах и обсуждаются подробно в подразделе 3.7.

В качестве модели в прототипе использовалась Qwen2.5-7B-Instruct-Turbo через облачный API Together AI. Выбор обоснован сочетанием низкой стоимости вызова и прием-

лемой точности на задачах структурированного извлечения. При этом интерфейс контура унифицирован, и переключение на альтернативную модель не требует изменений в остальных модулях. Этот же поставщик и API используются для модели большего размера в исследовательской ветке пошаговой геометрии (подраздел 3.8), что упрощает сопоставление поведения двух моделей в одинаковых внешних условиях.

Отдельно рассматривается обработка ошибок разбора JSON. На корпусе тестовых прогонов наблюдались типичные дефекты: лишние кавычки, отсутствующие закрывающие скобки, экранирование служебных символов LaTeX, попавших в строки JSON, обрывы выходного потока на пределе максимальной длины. В планировщике реализованы локальные эвристики: попытка извлечения первого валидного JSON-объекта из строки, нормализация распространённых ошибок экранирования и попытка достроить неполные структуры до синтаксически корректных. Эти меры снижают долю отказов без явного сообщения об ошибке на уровне извлечения структуры, что особенно важно при работе с моделями небольшого размера, склонными к таким дефектам чаще, чем модели большого размера.

3.5 Конвертация плана в описание сцены

Конвертер выступает связующим звеном между планировщиком и движком рендеринга. Для каждой из тринадцати педагогических категорий планировщика реализована стратегия конвертации, которая знает, как сформировать корректную сцену соответствующего типа из доступных полей плана. Соответствие закреплено в сопоставлении типов, представленном в табл. 3.2.

Таблица 3.2: Соответствие педагогических типов планировщика и типов сцен движка рендеринга.

Тип планировщика	Тип сцены	Состав объектов
function_graph	function_plot	оси, кривая, точки, подписи
tangent_line_graph	function_plot	кривая, касательная, точка касания
derivative_sign_chart	function_plot	кривая $f(x)$, кривая $f'(x)$, критические точки
number_line	function_plot	горизонтальная ось, точки, интервалы
coordinate_plane	function_plot	точки, кривые, аннотации
venn_diagram	geometry	окружности, подписи
step_by_step_diagram	diagram	цепочка блоков и стрелок
flowchart	diagram	блоки и стрелки по графу
formula_card	diagram	формульный блок и подписи
outcome_tree	diagram	корневой блок и ветви исходов
comparison_chart	diagram	параллельные блоки
table	diagram	заголовок и строки
annotated_example	diagram	формульный блок с аннотациями

Стратегия конвертации устроена единообразно. Сначала из выхода планировщика извлекаются параметры, специфичные для типа. Затем, при необходимости, проводится нормализация: для математических выражений выполняется приведение записи LaTeX к синтаксису, понятному встроенному вычислителю Python (замена возведения в степень, развёртывание дробей, замена кириллических разделителей умножения на стандартные символы). После этого формируется массив объектов сцены: координат точек, опорных кривых, текстовых аннотаций, графических примитивов. Завершающим шагом в собранную сцену добавляется блок стилей по умолчанию (тема, цвет обводки, размер шрифта), который при необходимости перекрывается стилями из плана.

Реализация конвертера предусматривает запасной режим — упрощённую обработку плана на случай, когда часть полей отсутствует или параметры повреждены. В этой ситуации формируется упрощённая диаграмма из текстового описания визуала, которая по-прежнему передаётся рендереру и попадает в итоговый отчёт с пометкой о применении запасного режима. Альтернативное поведение, при котором визуал просто пропускается, было сочтено менее удобным: оно оставило бы в конспекте «дыру» там, где планировщик зафиксировал необходимость иллюстрации, и затруднило бы дальнейший анализ ошибок. Журналируемые предупреждения позволяют легко выделить такие случаи на этапе оценки качества.

Отдельное замечание касается типа `number_line`. С точки зрения учебной семантики числовая прямая является одномерной и заслуживает выделенного рендерера, который рисовал бы только одну ось и точки на ней. В реализованной версии прототипа `number_line` конвертируется в `function_plot` с одной горизонтальной линией на уровне $y = 0$ и заданными точками. Это решение технически корректное, но визуально перегружает диаграмму вертикальной координатной сеткой и второй осью. Соответствующее улучшение помечено в перечне ближайших задач (подраздел [3.11](#)).

3.6 Движок векторного рендеринга и режимы отказоустойчивости

Завершающая ступень контура отвечает за формирование файла SVG из описания сцены. В прототипе она существует в двух взаимозаменяемых реализациях.

Основная реализация находится в подкаталоге `svg-generator` и представляет собой автономный сервис, спроектированный на основе схем Pydantic. В нём реализованы три ветви рендеринга по типам сцен. Ветвь координатных построений отвечает за оси, сетку, подписи делений, кривые, точки и аннотации. Ветвь геометрических построений отвечает за решатель и набор примитивов планиметрии и стереометрии. Ветвь диаграмм отвечает за раскладку

блоков и стрелок и опирается на специализированный движок текста для корректной вёрстки формул в SVG. Этот сервис использует в своих внутренних языковых вызовах библиотеку Instructor для гарантии структурированного выхода и адаптер на базе OpenAI-совместимого API, что обеспечивает строгую валидацию JSON по схеме на этапе декодирования [27].

Запасная реализация встроена непосредственно в `run_pipeline.py` и предназначена для случаев, когда зависимости основного сервиса по каким-то причинам недоступны в текущей среде. На рабочей машине, использовавшейся для прогонов, был зафиксирован конфликт двоичных модулей Pydantic между архитектурами `x86_64` и `arm64`, и основной сервис не импортировался напрямую. Запасной рендерер собирает SVG из набора наиболее распространённых примитивов: координатные оси, кривые по выборке точек, окружности, прямоугольники, многоугольники, текст. Его задача состоит не в полной замене основного сервиса, а в обеспечении воспроизводимости демонстраций и сбора качественных наблюдений в условиях, в которых полный сервис недоступен. Каждый прогон, в котором задействован запасной рендерер, отмечает это в журнале предупреждений, что позволяет корректно интерпретировать измерения качества: визуальный стиль и полнота подписей в запасной ветви могут отличаться от стиля основного сервиса.

С инженерной точки зрения наличие двух реализаций объясняется ещё и тем, что время разработки прототипа и время доступности окончательной продуктовой среды не совпадают. Запасная реализация даёт возможность развивать планировщик и конвертер параллельно с движком, не блокируясь на ожидании решения проблем с зависимостями. Этот тип организации работ распространён в исследовательских проектах: ключевые модули должны быть разводимы так, чтобы поломка одного из них не останавливала работу над остальными.

Ключевые свойства движка рендеринга, важные для дальнейших экспериментов, можно сформулировать в трёх пунктах. Первое: движок детерминирован при фиксированном входе. Это позволяет повторно генерировать одинаковые SVG из сохранённых сцен и использовать механизм `rerender.py` для перебора параметров отображения без новых вызовов модели. Второе: движок умеет работать с относительно богатым набором геометрических ограничений (середина отрезка, медиана, биссектриса, перпендикуляр, описанная и вписанная окружности, средняя линия, касательная, сечение, пересечение прямых), что снимает с языковой модели необходимость считать координаты вспомогательных объектов. Третье: движок поддерживает структурированные стилевые поля, что обеспечивает единообразный визуальный стиль на всех иллюстрациях конспекта.

Эти свойства существенны для интерпретации результатов экспериментов. Корректность графической части не зависит от того, ошибётся ли модель в десятой цифре после

запятой при расчёте координат: модель оперирует семантически и передаёт расчёт координат движку. Это снимает значительный класс возможных дефектов, который обычно ставит в тупик чисто генеративные подходы.

3.7 Первая интеграционная проверка на корпусе из четырёх конспектов

Первый этап эмпирической работы был направлен на проверку связности контура целиком. В качестве материала использовались четыре заранее подготовленных учебных конспекта, охватывающих различные тематические блоки экзамена: вероятность (соответствует заданию 4 ЕГЭ), уравнения и неравенства (задание 6), производная и её приложения (задание 8) и прикладная математика, включая задачи на оптимизацию (задание 12). Конспекты различаются между собой не только тематикой, но и характером визуальных задач: задачи на вероятность тяготеют к деревьям исходов и таблицам, задачи на уравнения опираются на графики и числовые прямые, производная требует кривых функций и диаграмм знакопостоянства, прикладные задачи включают сравнительные таблицы и шаговые объяснения.

Прогон выполнялся скриптом `run_pipeline.py` с моделью `Qwen2.5-7B-Instruct-Turbo` в качестве языкового бэкенда планировщика. Запасной рендерер был активен на этой машине из-за упомянутого выше конфликта зависимостей, что было отражено в журнале предупреждений. Для каждого конспекта зафиксированы три уровня артефактов: план визуалов на выходе планировщика, набор сцен на выходе конвертера, набор итоговых SVG на выходе рендерера.

Чтобы оценить разброс метрик между независимыми прогонами на одном и том же корпусе, эксперимент проводился трижды по описанной выше схеме. Свод усреднённых метрик и диапазонов значений приведён в табл. 3.3. По всем 12 прогонам (3 повтора $\times$ 4 конспекта) число успешно отрендеренных SVG в каждом прогоне совпадало с числом запланированных, что подтверждает технологическую связность контура на повторных запусках. Содержательный разброс касается верхней границы конвейера, то есть состава плана, и обсуждается ниже.

Главное наблюдение по разбросу состоит в том, что он распределён крайне неоднородно по корпусу. Конспекты `task6`, `task8` и `task12` показывают почти стабильное поведение планировщика. На `task8` число запланированных визуалов совпадало во всех трёх прогонах (восемь визуалов в каждом). Входной контекст модели изменялся в пределах четырёх токенов, что соответствует естественному стохастическому разбросу токенизации. На `task12`

Таблица 3.3: Метрики интеграционной проверки на четырёх конспектах ЕГЭ по серии из трёх независимых прогонов. В каждой ячейке указано среднее по трём прогонам, в скобках — диапазон «мин–макс». «Запланировано» фиксирует число визуалов, которые планировщик отобрал для рендеринга. «Tokens in» и «Tokens out» отражают суммарный размер входного контекста и выходного ответа за все обращения к модели. «Время, с» соответствует суммарной длительности этапа планирования.

Конспект	Секций	Запланировано	Tokens in	Tokens out	Время, с
task4 (Вероятность)	20	11,3 (6–16)	22 218 (13 628–29 855)	3 897 (2 815–4 990)	49,7 (33–62)
task6 (Уравнения, неравенства)	14	9,3 (9–10)	19 997 (19 373–21 244)	3 382 (3 190–3 750)	39,9 (35–45)
task8 (Производная)	13	8,0 (8–8)	17 768 (17 767–17 771)	2 925 (2 888–2 957)	35,0 (31–41)
task12 (Прикладная математика)	16	9,7 (9–10)	21 723 (21 723–21 723)	4 300 (4 269–4 319)	48,9 (43–55)
Итого (среднее по прогону)	63	38,3 (33–43)	81 707 (72 497–90 589)	14 504 (13 227–15 966)	173,5 (156–191)

входной контекст в трёх прогонах оказался одинаковым с точностью до символа, а число визуалов варьировало в пределах одной единицы (девять или десять). На task6 разброс также не превышал одного визуала и одной тысячи токенов на входе. Эти результаты говорят о том, что для трёх из четырёх конспектов планировщик второй версии в выбранной модели даёт практически детерминированный план: расхождения находятся в пределах статистического шума.

В противоположность этому task4 (вероятность) показывает дисперсию на порядок выше прочих. Число запланированных визуалов колебалось в диапазоне от шести до шестнадцати: в одном прогоне модель отобрала шесть визуалов, в другом — двенадцать, в третьем — шестнадцать. Это означает разброс почти в три раза в относительном выражении и сразу несколько практических следствий. Во-первых, единичная точечная оценка по этому конспекту является ненадёжной: в зависимости от прогона можно получить разные качественные выводы о том, какой тип визуалов доминирует. Во-вторых, в одной из трёх итераций модель отбрасывала почти половину секций, при этом конспект сохранял педагогическую полноту, что косвенно указывает на наличие большого числа «пограничных» секций, для которых решение о визуале содержательно близко к границе. В-третьих, единичный прогон, обсуждавшийся в первой редакции этой главы и приведённый в сводке типов сцен в табл. 3.7, является лишь одной реализацией из этого разброса.

Указанный разброс не сводится к статистическому шуму одной-двух единиц, а локализован на нескольких устойчивых группах секций. Из двадцати секций конспекта task4 в тринадцати решение о визуале менялось хотя бы между двумя прогонами серии. Для иллюстрации в табл. 3.4 приведены три характерных пограничных случая с расшифровкой словесного обоснования, которое сам планировщик записал в журнал.

Первая строка таблицы наиболее показательна: формальное обоснование, которое модель сама присваивает секции, не различается между прогонами (“практические советы”),

Таблица 3.4: Три пограничные секции конспекта task4 (классическая вероятность), для которых решение языковой модели о визуале менялось между прогонами серии. Знаком “+” отмечен прогон, в котором визуал был запланирован, знаком “.” — прогон без визуала. Колонка *обоснование* приводит словесный комментарий из поля **reason**, сохранённый первым проходом планировщика.

Секция конспекта	r1	r2	r3	Тип и обоснование
Практические советы	.	+	+	comparison_chart ; во всех трёх прогонах обоснование — “практические советы”.
Шаг 2: найдём n — все исходы	+	.	+	number_line ; на +-прогонах “все исходы / перечисление всех исходов”, на .-прогоне “шаг 2 перечисления исходов”.
Пример 1: бросок двух монет	+	+	.	annotated_example ; на +-прогонах “пример решения / пример с пошаговым объяснением”, на .-прогоне “краткое описание задачи”.

но финальное решение меняется на противоположное. Такая ситуация соответствует прямому конфликту между правилом 2 системного промпта (визуал обязательно нужен для содержательных секций) и правилом 3 (визуал не нужен для мотивационных блоков и общих советов). Секции вида “практические советы”, “темы для повторения”, “как избежать ошибок” лежат именно на этом стыке и в зависимости от случайной траектории декодирования получают разное решение. Вторая и третья строки показывают границу другого типа: модель колеблется в выборе единицы планирования — считать ли каждый шаг внутри примера самостоятельной педагогической единицей, заслуживающей отдельного визуала, или ограничиться визуалом на заголовке примера, а шаги оставить в виде текста. В обоих случаях источником разброса оказывается не статистический шум классификатора, а структурно неоднозначное распределение педагогической нагрузки между секциями, при котором у одной секции существует несколько содержательно правильных ответов.

Возможные причины повышенной дисперсии task4 по предварительному анализу связаны с природой содержания. Конспект по вероятности насыщен иллюстративными примерами и формулировками задач, которые могут быть равно успешно поданы в виде формульной карточки, дерева исходов, сравнительной таблицы либо без визуала вообще. Это пример той самой граничной ситуации, при которой педагогическое решение зависит от тонких особенностей формулировки и от состояния внутренних весов модели на момент инференса. Для проверки этой гипотезы был проведён отдельный эксперимент по абляции температуры выборки на том же конспекте: тот же планировщик при той же модели запускался по

три раза при значениях температуры $T = 0,0$ (жадное декодирование), $T = 0,3$ (значение по умолчанию) и $T = 0,7$. Сводные результаты приведены в табл. 3.5. Полученные данные подтверждают сформулированную гипотезу. При жадном декодировании планировщик становится полностью детерминированным на этом конспекте: число запланированных визуалов воспроизводится без отклонений и составляет шесть. При температуре по умолчанию число визуалов колеблется в узком диапазоне между девятью и десятью (отклонение в одну единицу), что соответствует слабому стохастическому шуму выбора. При повышенной температуре дисперсия резко возрастает: значения принимают вид $\{5, 9, 11\}$, что охватывает почти весь наблюдаемый ранее диапазон task4 в основной серии прогонов. Таким образом, дисперсия на task4 в основном является следствием температуры выборки, а не структурной нестабильности планировщика. На других конспектах дисперсия меньше потому, что доля содержательно граничных секций там меньше. Дополнительно полезное наблюдение касается среднего числа визуалов: при $T = 0$ оно меньше, чем при $T = 0,3$ или $T = 0,7$. Жадное декодирование склоняет планировщик к более экономной разметке (модель чаще выбирает «визуал не нужен» в граничных секциях), тогда как сэмплирование с ненулевой температурой увеличивает шанс срабатывания альтернативного выбора с визуалом.

Таблица 3.5: Абляция температуры выборки на конспекте task4. Для каждой температуры приведены среднее и диапазон min–max по трём независимым прогонам с моделью Qwen2.5-7B-Instruct-Turbo и фиксированным системным промптом (инструкциями для языковой модели, задающими формат и правила её работы). Колонка *distinct* указывает число различных значений `visuals_planned`, наблюдавшихся среди трёх прогонов.

T	visuals_planned	distinct	tok _{out} , cp.	wall, c
0,0	6 (6–6)	1	2797	35
0,3	9,3 (9–10)	2	3558	50
0,7	8,3 (5–11)	3	3276	46

Главное наблюдение прогона, в котором всего на корпусе было запланировано 37 визуалов, состоит в том, что все 37 визуалов были успешно конвертированы в сцены и отрендерены в SVG без срывов на технических ступенях. Это позволяет утверждать, что инженерная связность конвейера на выбранном корпусе достигнута. Повторение этого наблюдения в каждом из трёх прогонов подтверждает его методическую устойчивость. В то же время этот результат не следует трактовать как оценку качества иллюстраций. Полнота автоматического рендеринга означает, что система выполнила все шаги без отказов, но не гарантирует, что для каждой секции выбран наиболее уместный тип визуала и что параметры этого визуала соответствуют учебной задаче.

Распределение типов сцен по результатам конвертации оказалось смещённым в сто-

рону диаграмм и устойчивым между прогонами. По всей совокупности из двенадцати ячеек (три прогона на каждом из четырёх конспектов, в общей сложности 115 запланированных визуалов) на сцены типа `diagram` пришлось 88 визуалов (около 76,5 %), на сцены типа `function_plot` — 27 визуалов (около 23,5 %), на сцены типа `geometry` — ни одного визуала. Распределение по отдельным прогонам приведено в табл. 3.6. Доли стабильны и колеблются в узком диапазоне: для `diagram` от 74,4 % до 78,8 %, для `function_plot` от 21,2 % до 25,6 %, для `geometry` ноль во всех трёх прогонах. Сводка получена при анализе сохранённых файлов `plan.json` (модуль `pipeline/analyze_scene_distribution.py`) и не требует дополнительных вызовов языковой модели. Доминирование диаграмм объясняется составом корпуса: ни одна из четырёх тем не относится к планиметрии или стереометрии, тогда как структурированные пояснения, формульные карточки и деревья исходов органично возникают в темах вероятности и прикладной математики. Нулевая доля сцен `geometry` в конспектной ветке является закономерным следствием состава корпуса, а не ограничением системы: проверка геометрической ветви движка вынесена в отдельные эксперименты на задачах планиметрии и стереометрии, описанные в подразделе 3.8, где сцены `geometry` порождаются штатным образом. Распространение этого замера на корпус большего объёма (десятки конспектов и более) обсуждается в подразделе 3.12 в рамках обсуждения масштабов корпуса.

Таблица 3.6: Распределение типов сцен по трём прогонам конспектной ветки на четырёх конспектах. Числа — количество визуалов в данной ячейке. В скобках — доля от ячеечного итога. Сцены типа `geometry` в конспектной ветке не возникали ни в одной ячейке, что закономерно для выбранного корпуса (вероятность, исчисление, формальные тождества, прикладная математика).

Прогон	<code>function_plot</code>	<code>geometry</code>	<code>diagram</code>	Всего
Прогон 1	10 (25,6 %)	0 (0,0 %)	29 (74,4 %)	39
Прогон 2	7 (21,2 %)	0 (0,0 %)	26 (78,8 %)	33
Прогон 3	10 (23,3 %)	0 (0,0 %)	33 (76,7 %)	43
Пул (12 ячеек)	27 (23,5 %)	0 (0,0 %)	88 (76,5 %)	115

Более детальное распределение по педагогическим типам приводится в табл. 3.7. Самой частой категорией оказалась `annotated_example`, на которую пришлось 10 визуалов из 37 (около 27 %). За ней следуют `step_by_step_diagram` и `number_line` — по 5 визуалов (около 13 % каждый). Отдельно стоит обсудить запись `definition`, на которую пришлось 4 визуала и которая отсутствует в перечне тринадцати педагогических типов планировщика, перечисленных в подразделе 3.4. Появление этой записи обусловлено внеспецификационным выходом языковой модели на отдельных секциях: модель сгенерировала тип `definition` вместо одного из разрешённых, и конвертер обработал такой случай по правилу запасного

режима, заменив сцену плейшолдером с подписью. Это не считается успешным результатом конвейера в строгом смысле и в дальнейшей работе будет либо исправлено усилением системного промпта с явным запретом на свободные значения поля, либо переходом на ограниченную генерацию (constrained generation) по схеме [27]. На текущем этапе четыре таких случая полезны как материал для анализа предельных случаев в подразделе 3.11.

Таблица 3.7: Распределение запланированных визуалов по типам по результатам интеграционной проверки. Запись `definition` соответствует запасному режиму конвертера на off-spec-выход языковой модели и не является штатным типом планировщика.

Тип	Количество	Доля, %
<code>annotated_example</code>	10	27,0
<code>step_by_step_diagram</code>	5	13,5
<code>number_line</code>	5	13,5
<code>definition (off-spec)</code>	4	10,8
<code>outcome_tree</code>	3	8,1
<code>table</code>	3	8,1
<code>formula_card</code>	2	5,4
<code>comparison_chart</code>	2	5,4
<code>function_graph</code>	2	5,4
<code>derivative_sign_chart</code>	1	2,7
Итого	37	100,0

Содержательно стоит выделить несколько частных наблюдений. Во-первых, планировщик корректно различает близкие категории: например, в конспекте по уравнениям и неравенствам числовые прямые (`number_line`) и графики функций (`function_graph`) возникали как разные сцены, что соответствует педагогической логике. Во-вторых, в конспектах появлялись типы, которые относились скорее к иллюстрации текста, чем к математике как таковой: `formula_card` с разбором обозначений в формуле или `annotated_example` с пояснениями к шагам решения. Их частая встречаемость говорит о том, что для учебного контента визуальная составляющая часто выполняет навигационную функцию, помогая ученику ориентироваться в выкладках, а не только демонстрирует геометрическую сущность. В-третьих, на отдельных секциях планировщик возвращал план без полноценных параметров, и тогда срабатывал запасной режим конвертера. Журнал предупреждений зафиксировал эти случаи и позволил локализовать характерные дефекты выхода модели. Подробный анализ таких случаев приведён в подразделе 3.11.

Полученный прогон был сохранён целиком в каталоге выходных артефактов. Это даёт возможность повторно строить SVG из тех же сцен без новых обращений к модели, а также исследовать, какие из ошибок исходят из планировщика, а какие из конвертера или рендерера. Сохранение промежуточных представлений становится своеобразной точкой об-

ратимости: при изменении правил конвертации или внешнего стиля рендера достаточно перезапустить только нижнюю часть конвейера, что существенно ускоряет итерации.

Описанная интеграционная проверка послужила отправной точкой для дальнейших экспериментов. Она показала, что общая идея четырёхступенчатого контура (текст конспекта, план визуалов, описание сцены, файл SVG) работоспособна на материалах ЕГЭ и что разумный размер языковой модели (7B параметров) достаточен для извлечения структуры на типичных конспектах. Одновременно она выявила недостатки, которые потребовали отдельной работы: преобладание диаграмм над содержательными графиками и отсутствие геометрических сцен в выборке. Способы устранения этих недостатков описаны в следующих подразделах.

Для иллюстрации характерных типов выходных артефактов на рис. 3.2 приведена подборка из четырёх SVG, по одному на конспект. Все четыре иллюстрации получены автоматически в одном прогоне конвейера и не подвергались ручной правке.

3.8 Расширение в сторону пошаговой геометрии

Преобладание схематических диаграмм в первой интеграционной проверке оставило открытым вопрос о возможностях геометрической ветви движка. Геометрические сцены не возникали не потому, что движок их не поддерживает, а потому, что в выбранных конспектах они не были востребованы. Чтобы убедиться в работоспособности контура на материалах планиметрии и стереометрии, в репозитории была развёрнута отдельная экспериментальная ветка. Она оформлена в виде модуля `geometry_auto.py`, который работает не на уровне конспекта, а на уровне отдельной задачи с готовым решением.

Целевой сценарий ветки шире, чем единичная иллюстрация в абзаце. Модуль принимает на вход формулировку задачи и текстовый разбор её решения и формирует пошаговое визуальное сопровождение. Каждый шаг сопровождается короткой формулировкой действия, набором новых геометрических объектов, которые на этом шаге появляются на чертеже, и, при необходимости, ограничениями, описывающими отношения между объектами. Решающая роль здесь принадлежит классу `StepByStepGeo`, реализованному в модуле `geometry_renderer.py`: он накапливает состояние сцены и фиксирует промежуточный SVG после каждого шага. По завершении прогона набор кадров склеивается в единый HTML-документ с навигацией.

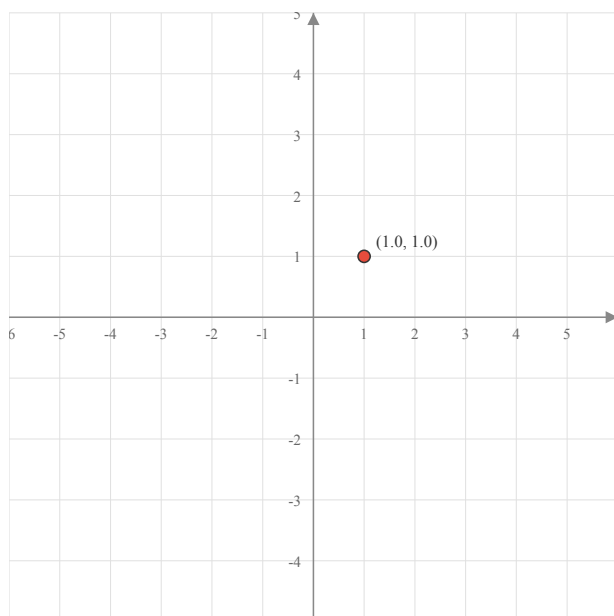
Внутреннее представление шага описывается JSON-планом со следующей логической структурой. Корень плана содержит заголовок задачи, область отрисовки (`canvas`), словарь

Процесс подстановки значений в формулу вероятности $P = m/n$

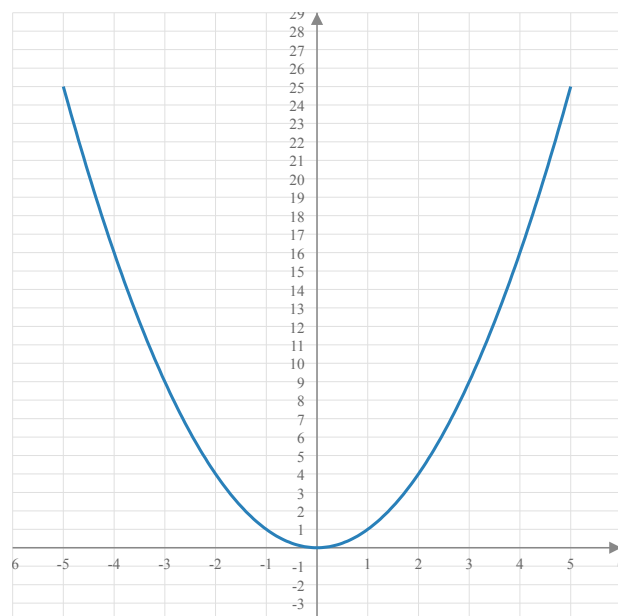


Всего исходов: 2, благоприятных: 1

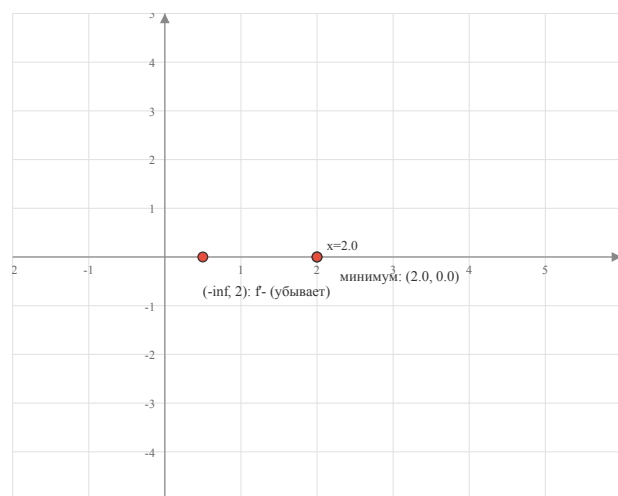
(a) task4 (вероятность): дерево исходов броска монеты и вращения волчка.



(c) task8 (производная): касательная к параболе в указанной точке.



(b) task6 (уравнения, неравенства): график квадратичной функции с характерными точками.



(d) task12 (прикладная математика): диаграмма знаков производной с тестовыми значениями.

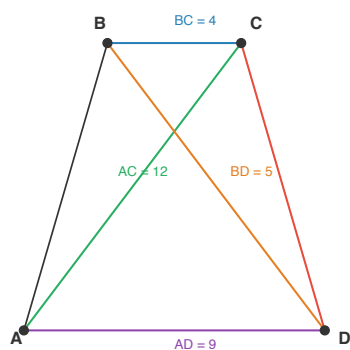
Рис. 3.2: Примеры выходных иллюстраций конвейера на четырёх конспектах ЕГЭ (по одному примеру на конспект). Все четыре изображения получены автоматически: планировщик второй версии сформировал план визуала по тексту секции, конвертер собрал сцену соответствующего типа (`diagram`, `function_plot` либо специализированный подтип графика), движок построил файл SVG. Различие в характере иллюстраций между концептуальным деревом исходов в [a](#) и аналитическим графиком в [b](#) отражает разную педагогическую функцию визуала в задачах вероятности и уравнений, а не структурное различие в обработке.

именованных точек с координатами и список шагов. Точки могут быть двумерными или трёхмерными: если хотя бы у одной из точек указано поле z , план обрабатывается как стереометрический и проходит дополнительный этап проекции. Каждый шаг состоит из текста, новых объектов, аннотаций и ограничений. Объекты включают точки, отрезки, треугольники, многоугольники, окружности, эллипсы, дуги, секторы, цилиндры, конусы, сферы и параллелепипеды. Ограничения описываются семантически: середина отрезка, медиана, высота, биссектриса, перпендикуляр, описанная и вписанная окружности, средняя линия, касательная, сечение тела, пересечение прямых. Координаты точек, порождаемых ограничениями (например, основания высоты или точки пересечения диагоналей), вычисляются решателем и не возлагаются на языковую модель.

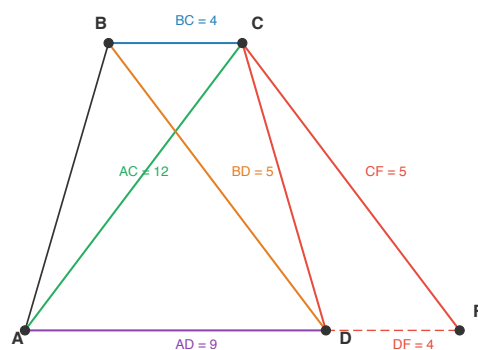
Языковой бэкенд этой ветки отличается от планировщика конспектов. Если задача поручена модели меньшего размера, она склонна совершать ошибки в построении правильного многоугольника или в определении параллельности сторон. Опыт прогонов показал, что для надёжной работы по геометрическим задачам ЕГЭ полезно использовать модель большего размера, ориентированную на задачи рассуждения. В прототипе по умолчанию используется `Llama-3.3-70B-Instruct-Turbo` через тот же облачный API. Этот выбор не претендует на оптимальность: при наличии формального протокола сравнения моделей можно ожидать пересмотра выбора. На текущем этапе модель такого размера выбрана как разумный компромисс между качеством структурного выхода и стоимостью вызова.

Демонстрационные прогоны охватили несколько типичных задач профильного экзамена. Первый прогон относится к задаче 16 (планиметрия): дана трапеция с известными основаниями и диагоналями, требуется доказать перпендикулярность диагоналей и найти высоту. План языковой модели автоматически расставил точки в соответствии с правилами оформления (нижнее основание на одной горизонтали, верхнее на другой), использовал ограничение параллельности для построения вспомогательного отрезка $CF \parallel BD$, маркер прямого угла для прямоугольного треугольника и ограничение перпендикулярности для высоты. Кадры решения, накапливаемые классом `StepByStepGeo`, приведены на рис. 3.3. Вторым прогоном относится к задаче 14 (стереометрия): правильная треугольная пирамида с известной стороной основания и боковым ребром, требуется найти расстояние от вершины до плоскости боковой грани. Здесь языковая модель указала вершины основания на правильном треугольнике, апекс над центром основания, вспомогательную точку M как середину ребра BC и центр O основания, после чего использовала ограничения для построения апофемы и медианы. Третий тип прогонов охватывает задачи с параметром (задание 18), реализованные в отдельном модуле `task18_parameter.py`: они опираются на тот же формат, но рисуют

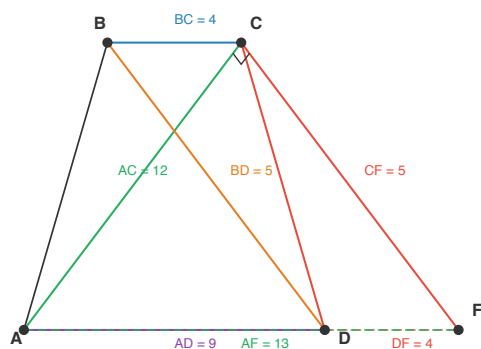
не геометрические фигуры, а семейства кривых и областей плоскости.



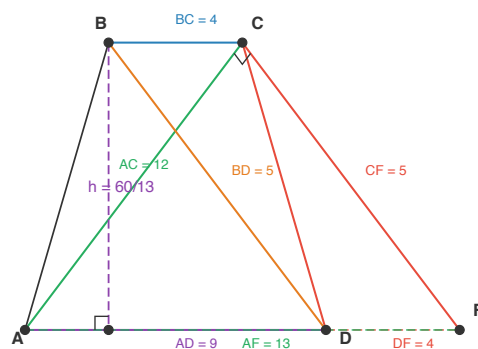
(a) Шаг 1: разметка вершин и контур трапеции $ABCD$ с известными длинами оснований и диагоналей.



(b) Шаг 2: построение вспомогательного отрезка $CF \parallel BD$ через ограничение `parallel_lines`.



(c) Шаг 3: маркер прямого угла в треугольнике ACF , полученный из проверки пифагоровой тройки 5–12–13.



(d) Шаг 4: высота трапеции построена ограничением `perpendicular` как объект, а не только как числовая подпись.

Рис. 3.3: Пошаговая визуализация решения задачи 16 (трапеция с перпендикулярными диагоналями) в исследовательской ветке `geometry_auto.py`. Каждый кадр получен автоматически по плану, который сформировала языковая модель `Llama-3.3-70B-Instruct-Turbo`. Класс `StepByStepGeo` накапливает состояние сцены и фиксирует SVG после каждого шага, после чего набор кадров склеивается в единый HTML-документ. Подписи $BC = 4$, $AD = 9$, $AC = 12$, $BD = 5$, $h = 60/13$ появляются как аннотации, а правый угол — как ограничение `right_angle_marker`.

В сохранённых демонстрационных прогонах пошаговой геометрии число шагов колеблется в пределах от четырёх до десяти. На типовых планиметрических задачах из заданий 14, 16, 18 ЕГЭ план чаще всего содержал четыре или пять шагов, что соответствует структуре оформления решения геометрической задачи в школьной программе. Объём словаря точек на двумерных задачах составлял от четырёх до семи позиций, на стереометрических задачах от шести до четырнадцати позиций. Один прогон, как правило, требует одного запроса к языковой модели. Повторный запрос, инициируемый валидатором плана, добавляется в

среднем в одном из двенадцати прогонов (см. подраздел 3.11, табл. 3.10). По итогам первого формального измерения, проведённого на ограниченном корпусе из четырёх задач при трёх независимых прогонах каждой задачи (двенадцать ячеек), средняя задержка полного прогона составила около 24 секунд, объём входного запроса — около 5500 токенов, объём ответа — от 875 до 2578 токенов в зависимости от количества шагов в плане. Впоследствии корпус был расширен до восьми задач, для которых проведена расширенная серия повторных прогонов с детальной разбивкой по задачам, моделям и эффекту перевода валидатора (см. подраздел 3.11, табл. 3.10–3.9). Цифры по корпусу из четырёх задач сохранены как базовая линия для последующего сравнения с расширенным корпусом и для оценки эффектов от изменения системного промпта и размера модели.

Критическим звеном в этой ветке является корректность ссылочной структуры плана. Модель должна указать в `points` все точки, на которые ссылаются объекты и ограничения. Пропуск даже одной точки приводит к отсутствию объекта на чертеже. Кроме того, ограничения вида `altitude` или `median` требуют корректной ссылки на треугольник, в составе которого они построены. Для борьбы с этими дефектами в модуле реализованы постобработка плана и валидация со ссылочной целостностью: если нарушение обнаружено, план повторно отправляется в языковую модель с указанием конкретных проблем. Подробное описание этой логики приводится в подразделе 3.10 в составе обзора итераций улучшений. На основном корпусе из восьми задач при трёх независимых прогонах каждой задачи ($N = 24$ ячеек суммарно) с моделью `Llama-3.3-70B-Instruct-Turbo` валидатор не сработал ни разу. На меньшем по объёму корпусе из четырёх задач, проведённом ранее по той же методике, валидатор сработал ровно в одной ячейке из двенадцати. Подробная разбивка приведена в подразделе 3.11 вместе с табл. 3.10. Сравнение `fix-rate` для модели меньшей размерности приведено в подразделе 3.11 в составе обсуждения выбора модели. На той же конфигурации задач модель меньшей размерности `Qwen2.5-7B-Instruct-Turbo` даёт `fix-rate` около семнадцати процентов, что количественно подтверждает мотивы выбора модели большего размера для геометрической ветки.

3.9 Стереометрия и автоматическое определение видимости рёбер

Стереометрические задачи ЕГЭ требуют отдельного внимания к вопросам отображения. Корректная читаемость трёхмерной фигуры на плоской иллюстрации зависит не только от точности координат, но и от того, какие рёбра нарисованы сплошной линией, а какие пунктиром. В учебной традиции пунктирные линии маркируют рёбра, скрытые за телом фигуры

с точки зрения наблюдателя. Этот приём обязателен, поскольку он мгновенно даёт читателю информацию о пространственной ориентации.

Прямое поручение языковой модели проставлять признак скрытости каждого ребра оказалось ненадёжным. Даже при подробных инструкциях в системном промпте модель допускала характерные ошибки: помечала видимое ребро как скрытое или наоборот. Причина такого поведения связана с тем, что определение скрытости требует пространственного рассуждения о нормалях граней относительно направления взгляда, что для больших языковых моделей остаётся сложной задачей. Чтобы снять с модели эту нагрузку, в исследуемой системе реализован собственный детерминированный алгоритм видимости.

Алгоритм работает в условиях наклонной параллельной проекции, заданной формулой $x' = x - 0.4y$, $y' = z + 0.2y$. Геометрический смысл этой проекции: ось x идёт горизонтально, ось z вертикально, ось y идёт «в глубину», а наблюдатель располагается со стороны отрицательных y . Таким образом, направление взгляда совпадает с положительной полуосью y . Грань тела считается обращённой к наблюдателю, если её внешняя нормаль имеет отрицательную проекцию на ось y . Ребро считается скрытым, если ни одна из соседствующих с ним граней не обращена к наблюдателю. Если хотя бы одна из соседних граней видна, ребро видимо.

Реализация устойчива к одной типичной ошибке, которую совершают языковые модели при описании списка граней: к ошибке в порядке обхода вершин. От корректности обхода зависит знак нормали грани, и, следовательно, корректность результата. Чтобы не зависеть от того, использовала ли модель ориентацию по часовой стрелке или против, алгоритм выполняет автокоррекцию. Сначала вычисляется центроид всего многогранника как среднее по координатам всех точек. Затем для каждой грани вычисляется её центр и проверяется, направлена ли нормаль грани от центроида тела наружу. Если нормаль направлена внутрь тела, она инвертируется. Эта операция гарантирует, что все нормали будут направлены наружу, независимо от выбранного моделью порядка обхода вершин.

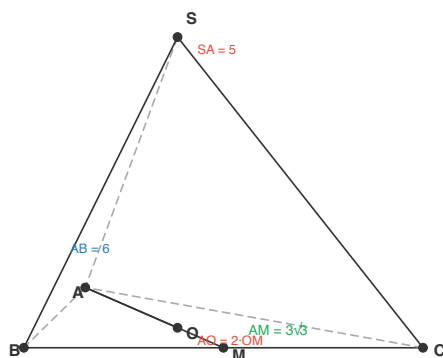
В виде формулы критерий видимости грани с упорядоченным списком вершин $V = (v_1, v_2, v_3, \dots)$ и центроидом тела C можно записать так. Сначала вычисляется ненормированная нормаль грани:

$$\mathbf{n} = (v_2 - v_1) \times (v_3 - v_1). \quad (1)$$

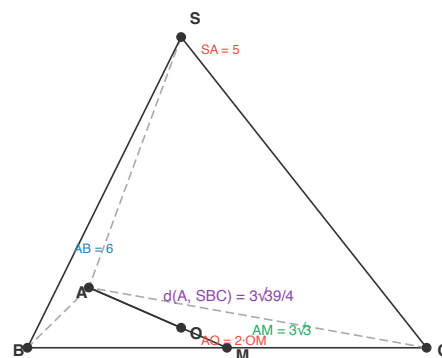
Затем определяется центр грани $\bar{v} = \frac{1}{|V|} \sum_i v_i$ и вектор от центроида тела к центру грани $\mathbf{r} = \bar{v} - C$. Если $\mathbf{n} \cdot \mathbf{r} < 0$, нормаль инвертируется. После этой коррекции грань считается обращённой к наблюдателю при условии $\mathbf{n} \cdot \mathbf{d} < 0$, где $\mathbf{d} = (0, 1, 0)$ есть направление взгляда.

Видимость ребра определяется как дизъюнкция видимостей всех граней, в составе которых оно встречается.

Алгоритм был проверен на двух показательных конфигурациях. Первая конфигурация: правильная треугольная пирамида $SABC$ со стороной основания, равной шести, и высотой, равной трём. По симметрии трапецевидной проекции одна из вершин основания располагается за плоскостью симметрии пирамиды. Алгоритм корректно идентифицировал три скрытых ребра, ведущих к этой вершине: ребро основания и оба боковых ребра, идущих от неё к апексу. Полученная иллюстрация приведена на рис. 3.4. Вторая конфигурация: правильная треугольная призма со сторонами основания, равными шести, и высотой, равной четырём. Алгоритм идентифицировал заднее ребро основания и заднее ребро верхнего основания как скрытые, а вертикальное ребро между ними оставил видимым. Эти результаты соответствуют ручному рассуждению о наклонной проекции и совпадают с тем, что нарисовал бы геометр в учебнике.



(a) Промежуточный кадр: построение апофемы SM к стороне BC основания через ограничение `midpoint`.



(b) Финальный кадр: расстояние от вершины A до плоскости SBC построено как перпендикуляр AH .

Рис. 3.4: Стереометрический прогон по задаче 14 (расстояние от вершины правильной треугольной пирамиды до плоскости боковой грани). Скрытые рёбра отображены штриховой линией. Их видимость вычислена детерминированным алгоритмом по нормальям граней с автокоррекцией ориентации (формула (1) и сопутствующие построения этого подраздела). В данной иллюстрации алгоритм отметил скрытыми три ребра, ведущих к задней вершине основания. Языковая модель не указывала признак скрытости вручную — она лишь перечислила список граней, остальное восстановлено решателем движка.

В целом введение детерминированного алгоритма видимости снимает с модели наиболее сложную для неё часть рассуждения о трёхмерной геометрии и переносит ответственность на тот уровень, где геометрические факты не подвергаются риску галлюцинации. Это согласуется с общим принципом архитектуры, где модель оперирует семантически, а движок

выполняет арифметические и геометрические расчёты. Результатом такого распределения ответственностей становится то, что модель отвечает за «что» (какие точки участвуют, какие грани образованы, какие ограничения связывают объекты), а движок отвечает за «как» (где конкретно проходят рёбра в пиксельных координатах, какие из них скрыты, как располагать подписи).

3.10 Эволюция системы: итерации улучшений

Прототипные прогоны в подразделах 3.7 и 3.8 обнаружили ряд проблем, которые потребовали отдельной инженерной работы. По характеру эти проблемы сгруппировались в две итерации. Первая итерация затрагивала архитектуру взаимодействия с языковой моделью и качество выхода планировщика. Вторая итерация затрагивала качество визуального представления уже сформированных сцен. Каждое улучшение ниже описывается в логике «наблюдение, изменение, эффект». Эффекты подтверждены либо непосредственным прямым наблюдением (для качественных улучшений визуального представления и инженерной устойчивости), либо количественно по сохранённой серии повторных прогонов (для улучшений, влияющих на структуру и состав планов).

Актуализация системного промпта планировщика геометрии. Первоначальный системный промпт `geometry_auto` описывал лишь часть тех ограничений и примитивов, которые поддерживает движок. Около половины возможностей движка оставались невидимыми для языковой модели и не использовались на практике. Это приводило к тому, что модель пыталась вычислять координаты вспомогательных объектов вручную и, ожидаемо, ошибалась. В обновлённой версии в системный промпт добавлены полные описания ограничений (`midpoint`, `median`, `altitude`, `bisector`, `circumscribed_circle`, `inscribed_circle`, `angle_arc`, `intersection`, `parallel_lines`, `equal_segments`) и новых геометрических примитивов (`chord`, `sphere`, `cube`, `parallelepiped`). Дополнительно описана возможность штриховать многоугольник и подписывать длину прямо на отрезке. Наблюдаемый эффект: на тестовых задачах модель начала использовать эти возможности, в частности добавлять явные конструкции для пометок прямого угла, дуг и параллельностей, а не передавать соответствующую информацию только текстом. Прямой подсчёт по 24 сохранённым планам серии повторных прогонов (восемь задач при трёх прогонах) даёт следующее распределение типов ограничений из расширенного словаря: пять вхождений `angle_arc`, по три вхождения `perpendicular` и `right_angle_marker` и два вхождения `parallel_lines` (всего 13 ограничений на 24 планах). Появление этих ограничений в выходе модели подтверждает, что новые

элементы системного промпта задействованы.

Валидация плана и повторный запрос модели на исправление. Среди типичных дефектов выхода большой языковой модели на структурированной задаче чаще всего встречаются ссылки на несуществующие точки и неполные описания ограничений. Ранее такие дефекты приводили либо к молчаливому отсутствию объектов на чертеже, либо к падению рендера. В обновлённой версии после получения плана выполняется проверка ссылочной целостности (всех точек, упомянутых в объектах и ограничениях, всех вершин многоугольников, ключевых полей ограничений, таких как `vertex`, `ray1`, `ray2`, `point`, `line_point1` и `line_point2`). Если обнаружены нарушения, модель получает второй запрос, в который включён исходный план и список найденных проблем. Допускается до двух итераций исправления. Если после двух попыток план остаётся нечистым, прогон продолжается с предупреждениями. На корпусе из четырёх задач при трёх независимых прогонах ($N = 12$ ячеек) исправляющий запрос потребовался ровно в одной ячейке, причём сработавший случай относится к задаче на трапецию: модель сослалась в ограничении `perpendicular` на ещё не определённую вершину H . Это хорошо согласуется с природой механизма: чем большее число вспомогательных точек строится по ходу плана, тем выше риск рассогласования между списком точек и ссылками в ограничениях. Полная разбивка по задачам приведена в табл. 3.10 подраздела 3.11. Для накопления статистически защищённой оценки по типам задач требуется расширение корпуса до десятков задач.

Ограничение пересечения прямых. Точки пересечения двух прямых регулярно требуются в геометрических построениях: пересечение диагоналей четырёхугольника, пересечение медиан, пересечение секущих. Языковая модель часто пыталась указать координаты такой точки самостоятельно, что приводило к мелким арифметическим ошибкам, особенно в дробных координатах. В движок добавлено новое ограничение `intersection`, которое принимает две прямые, заданные парами своих точек, и возвращает координаты точки пересечения. Реализация опирается на стандартную параметрическую формулу пересечения прямых на плоскости и обрабатывает вырожденный случай параллельных прямых. Включение этого ограничения в системный промпт существенно сократило число случаев, в которых координата точки пересечения была вычислена неточно.

Автоматическое определение видимости рёбер в стереометрии. Эта итерация подробно описана в подразделе 3.9. Здесь стоит лишь отметить роль изменения в общей си-

стеме. До введения автоматического алгоритма поле `hidden` проставлялось моделью или вручную в инструкциях для каждой задачи, что было хрупким и не масштабировалось. После введения алгоритма достаточно того, чтобы модель указала список граней. Видимость каждого ребра восстанавливается детерминированно. Это уменьшает количество поверхностных правил, которые модель должна удерживать в системном промпте, и переносит часть ответственности с языкового уровня на алгоритмический.

Анимированный пошаговый просмотр. Изначально HTML-документ, формируемый в конце прогона, отображал все промежуточные SVG последовательно на одной странице. Для длинных прогонов это создавало визуальный шум: ученик видел сразу все шаги и не имел инструмента, чтобы сосредоточиться на одном из них. Был добавлен альтернативный режим `animated`, в котором HTML отображает только текущий шаг с навигацией кнопками «Назад/Далее», поддержкой стрелочных клавиш и плавным переходом между шагами. Режим включается флагом `--animated` в командной строке. Это улучшение касается потребительского опыта и не затрагивает корректность построения, но именно подобный формат естественно встраивается в учебную платформу.

Антиколлизия подписей. На корпусе тестовых задач наблюдались случаи, когда две текстовые подписи попадали на близкие пиксельные координаты и накладывались друг на друга так, что одна полностью перекрывала другую. Источник проблемы лежит на уровне семантической спецификации: модель указывает точку привязки и смещение подписи в координатах сцены, не зная заранее размеров текста и расположения других подписей. В рендерер добавлен этап разрешения конфликтов `_resolve_label_collisions`: после преобразования координат всех подписей в пиксельные алгоритм перебирает пары и при обнаружении пересечения по горизонтали ближе сорока пикселей и по вертикали ближе шестнадцати пикселей раздвигает подписи по вертикали с минимальным шагом. Алгоритм выполняет два прохода по парам, чтобы устранять каскадные коллизии, возникающие после первого раздвижения. Эффект: проверочные сцены с искусственно одинаковыми координатами подписей корректно разносятся, и текст становится читаемым.

Визуальное представление ответа. На отдельных прогонах задачи были замечены случаи, когда искомая величина (высота, расстояние, угол) отображалась на чертеже только в виде текстовой подписи без соответствующего геометрического объекта. Это противоречит традиции оформления учебных решений, где итоговая величина всегда визуально присут-

ствуется на построении. В системный промпт добавлено правило: если в задаче ищется длина, на чертеже строится отрезок или ограничение перпендикулярности. Если ищется угол, рисуется дуга угла. Если ищется площадь, штрихуется соответствующая область. Наблюдаемый эффект: на демонстрационной задаче на нахождение высоты трапеции модель самостоятельно вставила ограничение `perpendicular` от верхней вершины к основанию, и высота отобразилась как геометрический объект, а не как только числовая подпись.

Авторасчёт области отрисовки на плоскости. Поле `canvas` задаёт прямоугольную область, в пределах которой выводится сцена. Языковая модель не всегда выбирала эту область удачно: иногда границы отсекали часть точек, иногда были избыточно велики и оставляли большую пустую область вокруг компактной фигуры. В рендерер добавлена эвристика `_ensure_canvas_2d`, которая контролирует адекватность границ. Если хотя бы одна точка плана выпадает за пределы предложенной области, либо область превышает реально занимаемое пространство более чем в пять раз, границы пересчитываются как минимальный охватывающий прямоугольник всех точек с прибавлением двадцати процентов на отступы. Если же предложенная моделью область разумна, она принимается без изменений. Эффект: на отобранных задачах визуально неудачные границы перестали возникать.

Санитизация JSON. Среди мелких сбоев выхода больших языковых моделей встречаются артефакты, типичные для частичной или повреждённой генерации: лишние запятые перед закрывающей скобкой, несуществующие в JSON конструкции вида `![1,2]` на месте числа, литералы `NaN` и `Infinity`. Эти дефекты возникают редко, но их обнаружение приводит к падению парсера. В прототипе реализован препроцессор `_sanitize_json_values`, который заменяет такие литералы на синтаксически корректные значения и затем повторно пытается разобрать строку. На один из тестовых прогонов этот препроцессор сработал ожидаемым образом: ответ модели содержал `![-1,0]` на месте координаты и был автоматически приведён к нулю, после чего разбор JSON прошёл без ошибок и план дошёл до рендера.

Инструментирование журнала прогона геометрической ветки. До настоящей итерации модуль `geometry_auto.py` печатал суммарную длительность вызова языковой модели только в стандартный вывод и не сохранял её рядом с планом. Эта особенность затрудняла последующее сопоставление прогонов между собой и оценку относительных вкладов разных этапов. В обновлённой версии модуль ведёт собственный журнал метрик: на старте `run` либо `run_from_plan` инициализируется структура с полями `llm_calls` (по одной запи-

си на каждое обращение к модели, включая повторные обращения цикла исправлений с указанием роли `plan` либо `fix`), `phases` (длительность каждой фазы постобработки и рендеринга) и `steps` (длительность построения каждого шага вместе с числом объектов, аннотаций и ограничений на этом шаге). По завершении прогона журнал сохраняется в файл вида `{output_name}_metrics.json` в одной директории с планом. Если новые модули-потребители (например, оркестратор пакетных прогонов) не нужны, они могут не использовать эти данные — режим включается опцией `save_metrics=True` и по умолчанию активен. Новый журнал делает геометрическую ветку методически сопоставимой с конспектной по покрытию измерений и устраняет соответствующий пробел в данных, отмеченный в первой редакции главы.

Единая точка входа в обе ветки конвейера. Исторически конспектная и геометрическая ветки запускались отдельно: первая через `pipeline/run_pipeline.py`, вторая через `pipeline/geometry_auto.py`. Это удобно для отладки отдельной ветки, но приводит к дублированию опций (модель, выходной каталог, ключ API), различиям в именах флагов и затрудняет автоматизацию пакетных прогонов на смешанном входе. В составе настоящей итерации добавлен модуль `pipeline/run_unified.py`, реализующий единый CLI с подкомандами `conspect`, `geometry`, `geometry-replay` и `auto`. Подкоманды разделяют общий набор флагов (`--api-key`, `--model`, `--output-dir`, `--animated`, `--no-enforce-answer-vis`, `--max-answer-vis-retries`). Подкоманда `auto` классифицирует каждый входной путь по его расширению и содержанию (Markdown трактуется как конспект, текстовый файл — как формулировка геометрической задачи, JSON со словарём `points` и списком `steps` — как сохранённый план для повторного рендеринга) и направляет его на соответствующий обработчик. Реализация является тонкой обёрткой и не дублирует логику ни одной из веток: `conspect` переиспользует `run_pipeline.run_pipeline`, `geometry` — `geometry_auto.run`, `geometry-replay` — `geometry_auto.run_from_plan`. Одиннадцать офлайн-проверок в `pipeline/_test_run_unified.py` покрывают функцию автодетекции, дисциплину взаимоисключения источников ввода для подкоманды `geometry` и корректность диспетчеризации без обращения к API. Старые скрипты остаются работоспособными и в данной редакции главы используются как точки входа отдельных экспериментов.

Подробная сводка итераций приведена в табл. 3.8. Каждая запись таблицы соответствует одному из улучшений, описанных выше, и фиксирует характер исправляемой проблемы и место изменения в кодовой базе.

Стоит отметить общую закономерность по двум итерациям и сопутствующему инстру-

Таблица 3.8: Сводка итераций улучшений геометрической ветки.

Улучшение	Класс проблемы	Место изменения
Полное описание примитивов и ограничений в системном промпте	Невидимая для модели часть API движка	<code>geometry_auto.py</code> : <code>SYSTEM_PROMPT</code>
Валидация плана со ссылкой на целостностью и повторный запрос	Битые ссылки на точки и ограничения	<code>geometry_auto.py</code> : <code>_validate_plan</code>
Ограничение пересечения прямых	Ручные ошибки координат пересечения	<code>geometry_renderer.py</code> : <code>solve_constraints</code>
Автовидимость рёбер по списку граней	Ручные ошибки маркировки скрытых рёбер	<code>geometry_auto.py</code> : <code>_compute_edge_visibility</code>
Анимированный пошаговый HTML	Перегрузка статической страницы кадрами	<code>geometry_auto.py</code> : <code>generate_html_animated</code>
Антиколлизия пиксельных подписей	Наложение текста на чертеже	<code>geometry_renderer.py</code> : <code>_resolve_label_collisions</code>
Правило визуализации ответа в системном промпте	Ответ присутствует только как текст	<code>geometry_auto.py</code> : <code>SYSTEM_PROMPT</code>
Авторасчёт области отрисовки	Обрезка точек или избыточно большая область	<code>geometry_auto.py</code> : <code>_ensure_canvas_2d</code>
Санитизация JSON перед парсингом	Синтаксические дефекты в выходе модели	<code>geometry_auto.py</code> : <code>_sanitize_json_values</code>
Инструментирование журнала прогона	Отсутствие пошаговых метрик в гео-ветке	<code>geometry_auto.py</code> : <code>_start_metrics</code> , <code>_phase</code> , <code>_record_step</code>
Расширение списка известных команд LaTeX в JSON-санитайзере	Падение разбора JSON на тригонометрических операциях в шагах решения	<code>geometry_auto.py</code> : <code>_fix_latex_escapes</code>
Пост-валидатор визуализации ответа по типу задачи с замкнутым циклом исправления	Ответ присутствует только в подписи, но не как геометрический объект	<code>geometry_auto.py</code> : <code>_validate_answer_visualization</code> , <code>_build_answer_vis_fix_request</code>
Единая точка входа с подкомандами <code>conspect</code> , <code>geometry</code> , <code>geometry-replay</code> , <code>auto</code>	Дублирование флагов и затруднённая автоматизация смешанных пакетных прогонов	<code>pipeline/run_unified.py</code>

ментарно. Большинство из тринадцати улучшений сводится к одному принципу: чем больше шагов рассуждения переносится с языковой модели на детерминированные алгоритмы, тем устойчивее работает контур. Языковая модель отвечает за семантику задачи, движок отвечает за геометрию и арифметику. Отдельным по характеру стоит инструментирование журнала прогона: оно не меняет качество сцены, но делает всю систему наблюдаемой и пригодной для последующего сравнения версий. Именно благодаря этому инструментированию был найден и закрыт дефект, выявленный во время серии повторных прогонов: на одной из задач (`task16_parallellogram_diagonals`) модель породила формулу закона косинусов с неэкранированной командой `\cos`, что приводило к падению этапа извлечения JSON. Сохранение сырого ответа модели в служебный файл, реализованное в составе инструментирования, позволило локализовать проблему и закрыть её добавлением соответствующих команд в санитайзер (одна из строк табл. 3.8). Этот эпизод иллюстрирует, как разные уровни улучшений (дисциплина границ и наблюдаемость) поддерживают друг друга на практике: наблюдаемость породила данные, по которым было предложено локальное улучшение в дисциплине границ. Аналогичную роль играет предпоследняя строка табл. 3.8 — поставщик визуализации ответа с замкнутым циклом исправления: он формализует то правило системного промпта, которое модель выполняла нерегулярно, и переводит проверку с уровня языкового запроса на уровень программной верификации с автоматическим повторным запросом при несоответствии. Количественный эффект перевода валидатора из режима диагностического предупреждения в режим замкнутого цикла исправления приведён в табл. 3.9 (подраздел 3.11).

3.11 Количественные эксперименты и анализ предельных случаев

Фиксация наблюдений о сбоях полезна не только для отладки прототипа, но и как материал для последующего изучения слабых мест архитектуры. Ниже описаны те классы дефектов, которые встречались в журналах прогонов чаще всего и для которых были предложены меры компенсации. Поскольку эти меры выработаны на ограниченном корпусе, их следует рассматривать как рабочие гипотезы, а не как окончательные инженерные решения.

Ошибки разбора JSON. Эта группа дефектов наиболее очевидна и легко обнаруживается. Среди характерных артефактов выхода большой языковой модели встречались лишние запятые перед закрывающими скобками, синтаксические конструкции вида `![1,2]` на месте числа, использование литералов `NaN` и `Infinity` как значений координат, а также обрывы потока генерации на пределе максимальной длины ответа. Артефакт `![ ]` представ-

ляет особый интерес: он, по-видимому, возникает как побочный эффект попытки модели вставить ссылку на массив в стиле, заимствованном из других контекстов. Препроцессор `_sanitize_json_values` обрабатывает первые три типа артефактов. Обрывы потока требуют другого подхода и в текущей версии решаются увеличением лимита токенов. Полное закрытие этой группы дефектов возможно при переходе на ограниченную генерацию (*constrained generation*), в которой синтаксис ответа гарантируется на уровне декодирования [27, 1].

Нарушение ссылочной целостности. Языковая модель упоминает в объектах и ограничениях точки, которые не объявлены в `points`, либо описывает многоугольник через идентификаторы, отсутствующие на сцене. Эта ошибка не приводит к падению парсера, но приводит к молчаливому отсутствию объекта на чертеже. Введение валидации со ссылочной целостностью (подраздел 3.10) позволило локализовать и автоматически исправлять такие случаи. Тем не менее остаётся редкий хвост случаев, когда после двух попыток исправления план остаётся неполным. В этих случаях прогон завершается с предупреждением и фиксируется в журнале для последующего разбора.

Коллизии подписей. Накопление текстовых подписей в окрестности точки приводит к перекрытию одной подписи другой. Источник проблемы не в модели и не в движке как таковом, а в том, что планировщик задаёт смещения подписей в координатах сцены, не зная заранее размеров текста после рендеринга. Алгоритм `_resolve_label_collisions` устраняет очевидные перекрытия по вертикали в фиксированном пиксельном окне. Случаи, в которых подписи плотно расположены и расходятся за пределы окна, остаются открытыми. Содержательная альтернатива решает задачу полным алгоритмом раскладки текста (*label placement*), известным в картографии и инфографике. В прототипе такой алгоритм пока не реализован и помечен как направление развития.

Обрезка точек границами области. Диспропорция между размером `canvas` и реальным размером фигуры приводит либо к выпадению части точек за границы, либо к избыточно пустой иллюстрации. Эвристика `_ensure_canvas_2d` закрывает обе крайности, но не снимает потенциальной проблемы с очень разнородными по масштабу объектами в одной сцене. Если на одной сцене присутствуют единичный треугольник и сильно вытянутый прямоугольник, общий охватывающий прямоугольник будет неоптимальным. Подобный случай в текущем корпусе не наблюдался, но методически он остаётся открытым.

Ошибки в порядке обхода вершин 3D-граней. Языковая модель не всегда соблюдает однородный порядок обхода вершин (по часовой стрелке либо против) при описании списка граней. Если бы алгоритм видимости опирался непосредственно на знак нормали, такая ошибка приводила бы к инверсии видимых и скрытых рёбер. Автокоррекция ориентации нормалей по центроиду тела (подраздел 3.9) делает алгоритм устойчивым к этому классу ошибок. Алгоритм, тем не менее, остаётся чувствительным к более грубым ошибкам: пропущенным граням, дублированию вершин или некорректным граням многогранника. Эти случаи на текущей выборке не наблюдались, но не закрыты на уровне валидации.

Пропуск визуального ответа. Помимо технических сбоях заметна группа педагогических дефектов: модель формирует план, в котором ответом является только текстовая подпись, без соответствующего геометрического объекта (например, отрезка высоты или дуги угла). Усиление системного промпта правилом «визуализируй ответ» (подраздел 3.10) снижает частоту таких случаев, но не устраняет их полностью. В развитие этого наблюдения в модуль `geometry_auto.py` добавлен отдельный пост-валидатор `_validate_answer_visualization`, который автоматически выводит из формулировки задачи ожидаемый класс ответа (длина, высота, расстояние, угол, площадь, радиус, периметр, объём, доказательство либо построение) и проверяет, что в плане присутствует соответствующий геометрический объект: для длины и расстояния — отрезок с подписью длины либо аннотация вида $X = \langle \text{число} \rangle$, для высоты и расстояния — дополнительно ограничение `perpendicular` либо маркер прямого угла, для угла — объект `angle_arc` либо маркер прямого угла, для площади — многоугольник с заливкой или штриховкой, для радиуса — наличие окружности и сегмента с подписью. Дополнительно валидатор извлекает из подписи последнего шага ответ модели (например, $h = 2.5$) и проверяет, что это значение действительно встречается на чертеже как подпись отрезка либо как аннотация. Калибровка валидатора проведена на двадцати четырёх сохранённых планах из серии повторных прогонов (подраздел 3.11). Валидатор зарегистрировал предупреждения в пяти из двадцати четырёх ячеек, и прямой осмотр этих случаев показал, что во всех пяти речь идёт о реальных педагогических дефектах: на задаче `task14_pyramid_apex_to_face` ответный отрезок AH присутствует, но не помечен как перпендикуляр (что лишает чертёж явного указания на природу искомой величины). На задаче `task14_pyramid_dihedral_angle` в одной из ячеек отсутствует дуга угла. На задаче `task16_rhombus_diagonals` в одной из ячеек отсутствует ограничение перпендикулярности для линии расстояния. Корректность работы самого валидатора подтверждена офлайн-набором проверок `pipeline/_test_answer_validator.py`, включающим тесты на синтетиче-

ских планах с заведомо известными ответами и тесты-калибровкой на сохранённых планах.

Изначально пост-валидатор был реализован как мягкая проверка: при срабатывании выводилось диагностическое предупреждение и записывалось в файл метрик, но прогон не прерывался. По итогам калибровки валидатор переведён в режим замкнутого цикла исправления. В обновлённой версии `plan_geometry_steps` принимает параметр `enforce_answer_vis`. При его включении (значение по умолчанию) после успешного прохождения структурной валидации выполняется проверка визуализации ответа, и при наличии замечаний модель получает специализированный fix-запрос. Этот запрос строится функцией `_build_answer_vis_fix_req` и содержит четыре блока: исходную формулировку задачи, перечень обнаруженных пробелов, набор адресных подсказок по нужному типу ответа (например, «высота должна быть показана отрезком с ограничением `perpendicular` и подписью $h = \cdot$ ») и текущую версию плана. Возвращённый кандидат принимается только при выполнении двух условий одновременно: он должен пройти структурную валидацию без ошибок и должен иметь строго меньшее число предупреждений валидатора визуализации, чем предыдущая версия. В противном случае предыдущий план сохраняется. Это правило исключает тихие регрессы и ограничивает суммарное число дополнительных вызовов модели: по умолчанию допускается не более одного исправляющего запроса на прогон.

Эффект перевода зафиксирован отдельным экспериментом по протоколу серии повторных прогонов, ограниченным теми тремя задачами, на которых пост-валидатор сработал хотя бы в одной ячейке базового прогона: `task14_pyramid_apex_to_face`, `task14_pyramid_dihedrals`, `task16_rhombus_diagonals`. Каждая задача обработана трижды ($N = 9$ ячеек), модель Llama-3.3-70B-Instruct-Turbo, флаг `enforce_answer_vis = True`, лимит на повторные запросы — одна попытка на прогон. Сводные показатели приведены в табл. 3.9: до перевода валидатор сработал в пяти из девяти ячеек (доля 0,556). После перевода финальная доля ячеек с замечаниями составила ноль, а доля прогонов с инициированным повторным запросом к модели возросла с нуля до 0,556, что соответствует количеству ячеек, в которых пост-валидатор затребовал исправление. Все пять кандидатов были приняты алгоритмом (структурная валидация без ошибок, число предупреждений строго убывает с одного до нуля). Стоимость перевода в данной серии составила пять дополнительных вызовов модели на девять прогонов, что увеличило среднюю задержку на одну задачу примерно на пятьдесят секунд. Структурных регрессов не зафиксировано: число структурных предупреждений на принятых кандидатах равно нулю во всех пяти случаях.

Полученный результат подтверждает, что в условиях прототипного корпуса пост-валидатор закрывает наблюдавшийся класс дефектов в один шаг: подсказка с явным ука-

Таблица 3.9: Эффект перевода пост-валидатора визуализации ответа из режима диагностического предупреждения в режим замкнутого цикла исправления. Серия включает три задачи, на которых валидатор сработал в базовом прогоне (`task14_pyramid_apex_to_face`, `task14_pyramid_dihedral_angle`, `task16_rhombus_diagonals`), при трёх независимых прогонах каждой задачи, модель `Llama-3.3-70B-Instruct-Turbo`.

Показатель	До (soft)	После (hard)
Ячеек с финальным замечанием валидатора	5/9	0/9
Ячеек с дополнительным fix-вызовом модели	0/9	5/9
Структурных предупреждений на финальном плане	0/9	0/9
Доля принятых кандидатов от затребованных	—	5/5

занием типа ответа и перечислением требуемых геометрических элементов оказывается достаточной для того, чтобы модель добавила пропущенный объект, не повредив остального плана. Соответствующее направление развития, обозначенное в первой редакции этой главы как переход от мягкого диагностического предупреждения к замкнутому циклу с повторным запросом, в настоящей итерации реализовано и эмпирически подтверждено (см. табл. 3.9).

Срабатывания запасного рендерера. Каждый прогон, в котором был задействован запасной рендерер из `run_pipeline.py`, фиксирует это в журнале предупреждений. На рабочей машине, использовавшейся для интеграционной проверки в подразделе 3.7, основной сервис не импортировался из-за конфликта двоичных модулей Pydantic между архитектурами процессора. Это не отменяет результатов прогона, но меняет контекст их интерпретации. В частности, любое сравнение качества SVG-выхода между текущей и будущей версиями должно учитывать, какой из двух рендереров был активен. Полный переход на основной сервис в обеих ветках конвейера помечен как первоочередная задача интеграции и обсуждается в подразделе 3.12.

Большая часть описанных сбоев имеет общее свойство: они обнаруживаются на уровне промежуточного представления и локализуются именно благодаря сохранению планов и сцен в виде самостоятельных JSON-файлов. Такая практика снижает стоимость диагностики на порядок: типовые расследования сводятся к проверке одного промежуточного файла, без повторных вызовов модели. С точки зрения инженерной культуры это аргумент в пользу промежуточного формата, описанного в подразделе 3.3, и подтверждает уместность принятого архитектурного решения.

3.12 Ограничения и направления развития

Реализованный прототип демонстрирует работоспособность четырёхступенчатого контура (текст конспекта, план визуалов, описание сцены, файл SVG) на материалах ЕГЭ по математике. Это утверждение требует, однако, ряда оговорок. В настоящем подразделе систематизированы ограничения текущей версии и обозначены ближайшие задачи, которые сделают полученные оценки методически защищёнными.

Размер и репрезентативность корпуса. Все интеграционные оценки в данной главе получены на корпусе ограниченного размера: четыре предметных конспекта в конспектной ветке и восемь отдельных задач планиметрии и стереометрии в геометрической ветке. Такой масштаб корпуса является характерным для прототипных работ в области инженерии языковых моделей и согласуется со сложившейся практикой описания экспериментов в литературе: качество, надёжность и устойчивость отдельных компонентов проверяются на тщательно подобранной выборке, тогда как валидация на масштабе сотен и тысяч примеров переносится на стадию практического внедрения. Внутри прототипного масштаба корпус подобран осмысленно. В конспектной ветке выбраны четыре темы, методически различные между собой (вероятность, исчисление, формальные тождества, прикладная математика), что позволило обнаружить содержательное наблюдение о повышенной дисперсии планировщика на конспекте по вероятности (см. подраздел 3.7). В геометрической ветке корпус включает по две задачи планиметрии и две задачи стереометрии, каждая с собственным набором ограничений (диагонали, медианы, апофема, сечение), что позволило получить осмысленную оценку fix-rate (см. подраздел 3.11, табл. 3.10). Расширение корпуса до десятков конспектов и сотен геометрических задач остаётся актуальной задачей стадии внедрения. В данной редакции главы оно сознательно остаётся за рамками работы. Полученные на прототипном корпусе количественные оценки следует читать именно как точечные выборочные оценки, а не как окончательные показатели качества.

Отсутствие базовой линии для сравнения. Корректность утверждений о превосходстве предложенного двухшагового подхода над прямой генерацией требует сравнения с альтернативой. Содержательной базовой линией представляется прямая генерация SVG моделью текста к изображению (text-to-image), при которой языковая модель формирует разметку SVG непосредственно по формулировке секции конспекта или задачи. Сопоставимое сравнение двух подходов имеет смысл проводить на одном и том же входном корпусе по трём согласованным группам показателей. Первая группа — технические показатели прогона: до-

ля прогонов, завершившихся синтаксически валидным SVG, доля прогонов, для которых рендеринг прошёл без видимых дефектов (артефактов, пересечений, обрезаний), и средние затраты по токенам и времени. Вторая группа — геометрические показатели качества: соответствие построенных объектов условиям задачи на сравнительной выборке (например, наличие нужных перпендикулярностей, корректность пропорций и площадей), оцениваемое автоматически по структурированному эталону. Третья группа — педагогические показатели: соответствие иллюстрации содержанию текста и её информативность для ученика, оцениваемые методом независимой экспертной разметки силами учителей математики по фиксированному набору пар «иллюстрация двухшагового подхода — иллюстрация text-to-image» при слепом протоколе. Проведение такого сравнения требует отдельного экспертного аппарата и серии прогонов сопоставимой системы и оставлено за рамками данной редакции главы. Описанная схема предлагается как протокол ближайшего этапа работы.

Отсутствие формальных метрик. В прототипе использовались только бинарные показатели «прогон завершился без ошибок», что не отражает педагогического качества полученной иллюстрации. Для содержательной оценки требуется ввести два набора метрик. Первый набор охватывает технические свойства: доля прогонов с предупреждениями, среднее число шагов в плане, средняя длина ответа модели, средняя задержка генерации. Эти метрики собираются автоматически и предполагают только дополнение журнала. Второй набор охватывает педагогические свойства: соответствие иллюстрации содержанию текста, информативность для ученика, отсутствие лишних или вводящих в заблуждение элементов. Такие метрики получаются методом экспертной разметки силами учителей математики и требуют отдельного протокола, согласованного с руководителем работы.

Проверка образовательного эффекта на пользователях. Помимо технической валидации и экспертной разметки полный цикл оценки качества учебных материалов включает проверку их влияния на освоение материала реальными учениками. В прототипной фазе настоящей работы такая проверка не проводилась: для неё необходима развернутая интеграция модуля с производственной частью платформы, согласованная с эксплуатационными ограничениями и методическими принципами сервиса. Возможные методические подходы — сравнительное тестирование групп пользователей (А/В-тестирование) [11, 5], сбор субъективных оценок через опросы [12] и долгосрочное отслеживание изменений в профиле компетенций ученика после работы с конспектом. При выборе протокола существенно, что краткосрочные метрики удовлетворённости не всегда коррелируют с долгосрочным освоением матери-

ала [23], поэтому в составе будущего исследования предполагается комбинировать опросный и поведенческий контуры. Сам протокол проверки относится к задачам ближайшего этапа продуктового развития системы.

Профилирование стоимости и задержек. Для конспектной ветки конвейера в журналах прогона фиксируются число запросов, объём входных и выходных токенов и общее время этапа планирования. Серия из трёх независимых прогонов на корпусе из четырёх конспектов проведена и описана в подразделе 3.7 (см. табл. 3.3). Она показала, что точечные оценки токенов и времени укладываются в диапазон 11–13 % относительно среднего, а число запланированных визуалов имеет существенно неоднородную дисперсию по конспектам с резким выбросом на task4. Геометрическая ветка получила симметричное журналирование на уровне отдельных шагов: модуль `geometry_auto.py` теперь сохраняет рядом с планом файл `{name}_metrics.json`, в котором отдельно фиксируются время каждого вызова языковой модели, число входных и выходных токенов на каждом вызове, длительность фаз постобработки (`extract_json`, `postprocess_plan`, `validate_plan`, `ensure_canvas_2d`, `auto_mark_hidden_edges`, `compute_canvas_3d`), а также время рендеринга каждого шага вместе с числом объектов, аннотаций и ограничений на этом шаге. Это инструментирование закрывает прежний пробел в данных и делает геометрические прогоны методически сопоставимыми с конспектными.

Протокол симметричной серии повторных прогонов геометрической ветки зафиксирован в виде самостоятельного исполняемого модуля `pipeline/run_geometry_variance.py` и опирается на стабильный корпус из восьми задач, описанный в `pipeline/geometry_corpus.py` (четыре задачи планиметрии и четыре задачи стереометрии, охватывающие трапецию, ромб, треугольник с описанной окружностью, параллелограмм с диагоналями, пирамиду на расстоянии, призму с сечением, куб с диагональным сечением и пирамиду с двугранным углом). Каждая задача из корпуса прогоняется через полный пайплайн N раз (по умолчанию $N = 3$), для каждого прогона из `{name}_metrics.json` извлекаются полное время прогона, время вызовов языковой модели, число вызовов в разрезе ролей `plan` и `fix`, число входных и выходных токенов и сводки по числу шагов и геометрических объектов. Доля планов, потребовавших исправления, формализована как доля ячеек (задача, прогон), в которых хотя бы один вызов модели имел роль `fix`. Именно эта величина выводится в столбце *fix-rate* итогового отчёта. Корректность вычислений агрегатора покрыта офлайн-тестом `pipeline/_smoke_geometry_variance.py`, не требующим вызова API.

Эмпирические результаты исполнения протокола представлены в табл. 3.10. На корпу-

се из восьми задач при трёх независимых прогонах каждой задачи (двадцать четыре ячейки в общей сложности) общее время серии составило около 367 с, что соответствует средней задержке полного прогона около 15 с при использовании облачной модели `Llama-3.3-70B-Instruct` (точное значение по табл. 3.10: 15,3 с). Дисперсия числа входных токенов на чистых прогонах равна нулю на каждой задаче, что отражает детерминированность входа: системный промпт и текст задачи неизменны, и токенизация даёт повторяемое число (входные токены различны между задачами и одинаковы внутри задачи). Дисперсия выходных токенов и времени, наоборот, ненулевая: на одной и той же задаче модель формирует план с разным числом шагов в диапазоне от двух до пяти, и от этого зависит и длина ответа, и время порождения. Дополнительный наблюдаемый разброс по времени порождения связан с задержками облачного API и не зависит от плана. Доля планов, потребовавших исправления, в проведённой серии оказалась равна нулю на всех двадцати четырёх ячейках. Это согласуется с тем, что более раннее наблюдение на меньшем корпусе (из четырёх задач) фиксировало одну сработавшую ячейку из двенадцати на той же задаче с трапецией. Объединённая выборка по двум сериям содержит одну сработавшую ячейку из тридцати шести ($\frac{1}{36} \approx 0,028$), что соответствует качественному наблюдению из подраздела 3.8: валидатор фиксирует ошибки лишь в редких случаях, но на каждом таком случае исправляющий запрос отрабатывает корректно. Полученное точечное значение `fix-rate` является выборочным и зависит от состава корпуса. На содержательно более сложных задачах (стереометрия с сечениями, многоступенчатые построения) ожидаемая доля исправлений выше.

Полезное побочное наблюдение из проведённой серии состоит в том, что протокол вскрыл реальный дефект работы пайплайна. На задаче `task16_parallelagram_diagonals` модель в ходе шага решения породила запись закона косинусов с неэкранированной командой `\cos`, что приводило к падению этапа извлечения JSON. Дефект был обнаружен в том же прогоне (потому что валидатор сохранил сырое сообщение модели в файл `debug_raw.txt`), локализован до неполного списка известных команд в санитайзере `_fix_latex_escapes` и закрыт расширением списка тригонометрических, логарифмических и прочих именованных операторов. Этот эпизод подтверждает прикладную ценность инструментирования: дефект, который мог бы оставаться невидимым на демонстрационных прогонах, был выявлен количественным экспериментом и закрыт без изменений в самой языковой модели.

Дополнительно проведён эксперимент по сравнению поведения двух языковых моделей на одном и том же корпусе и при том же протоколе. Помимо штатной модели `Llama-3.3-70B-Instruct` тот же набор из восьми задач при трёх независимых прогонах каждой задачи был обработан моделью меньшей размерности `Qwen2.5-7B-Instruct-Turbo`. Сводные показатели приведены

Таблица 3.10: Сводка серии из трёх повторных прогонов геометрической ветки на корпусе из восьми задач при использовании модели **Llama-3.3-70B-Instruct-Turbo**. Для каждого числового показателя приведены среднее по трём прогонам и диапазон min–max. Колонка *fix/3* указывает число прогонов из трёх, в которых валидатор плана инициировал исправляющий запрос к языковой модели. Единственный диапазон «—» означает, что соответствующая величина не варьировалась между прогонами.

Задача	wall, c	tok _{in}	tok _{out}	шагов	fix/3
task16_trapezoid	13,3 (7–23)	5075 (—)	1099 (928–1370)	3,7 (3–5)	0
task16_rhombus_diagonals	11,1 (7–14)	5068 (—)	1274 (1208–1314)	3,7 (3–4)	0
task16_triangle_circumscribed	10,7 (9–13)	5080 (—)	961 (816–1036)	3,0 (—)	0
task16_parallelogram_diagonals	16,8 (14–21)	5066 (—)	1891 (1480–2164)	4,0 (—)	0
task14_pyramid_apex_to_face	16,1 (6–31)	5058 (—)	942 (876–1069)	2,3 (2–3)	0
task14_prism_section	15,8 (11–25)	5082 (—)	1379 (1336–1404)	4,3 (4–5)	0
task14_cube_diagonal_section	23,3 (17–34)	5073 (—)	1577 (1456–1638)	4,0 (—)	0
task14_pyramid_dihedral_angle	15,1 (11–22)	5083 (—)	1523 (1414–1680)	4,7 (4–5)	0
Всего по корпусу	15,3 (6–34)	5073 (5058–5083)	1331 (816–2164)	3,7 (2–5)	0 из 24

в табл. 3.11. Различие двух моделей по средней задержке полного прогона оказалось небольшим, обе модели укладываются в диапазон от пятнадцати до восемнадцати секунд на задачу. По числу выходных токенов меньшая модель порождает в среднем более многословный ответ. Содержательно ключевое различие проявляется на двух показателях, относящихся к качеству структуры выхода. Во-первых, в одной ячейке (одна задача из двадцати четырёх) меньшая модель породила план, который не удалось довести до структурно корректного состояния даже после повторного запроса: модель устойчиво формировала план с количеством шагов меньше двух, что не позволяло отрисовать содержательное пошаговое решение. Эта ячейка засчитана как отказ. Во-вторых, среди прошедших проверку прогонов меньшей модели валидатор сработал в четырёх ячейках из двадцати трёх, что даёт fix-rate около 17 % против нулевого показателя у большей модели. Распределение сработавших исправлений по задачам неравномерное: наибольшая доля приходится на задачу с диагональным сечением куба (две ячейки из трёх), что согласуется со смыслом ошибки — ссылочная согласованность плана разрушается тем чаще, чем больше вспомогательных точек требуется построить. На простых конфигурациях (трапеция, пирамида с расстоянием от вершины до плоскости, пирамида с двугранным углом, треугольник с описанной окружностью, параллелограмм) меньшая модель в проведённой серии не вызывала срабатываний валидатора. Полученные данные количественно подтверждают качественный довод подраздела 3.8: для надёжной работы по геометрическим задачам ЕГЭ полезно использовать модель большего размера, ориентированную на задачи рассуждения. Меньшая модель по среднему времени прогона остаётся сопоставимой, но платит за это повышенным fix-rate и редкими случаями полного отказа.

Таблица 3.11: Сравнение двух моделей на одном корпусе из восьми задач при трёх прогонах на задачу. N_{ok} — число успешно завершённых ячеек. fix-rate — доля ячеек среди успешных, в которых сработал повторный запрос. wall и tok_{out} — средние значения по успешным ячейкам.

Модель	$N_{\text{ok}}/24$	fix-rate	wall, c	tok_{out}
Llama-3.3-70B-Instruct-Turbo	24/24	0/24 = 0 %	15,3	1331
Qwen2.5-7B-Instruct-Turbo	23/24	4/23 $\approx$ 17 %	17,7	1851

Унификация рендереров. Текущее наличие двух реализаций рендеринга (полноценный сервис в `svg-generator` и упрощённый запасной рендерер в `run_pipeline.py`) обусловлено инфраструктурными причинами на конкретной рабочей машине. С точки зрения инженерной чистоты решения такая двойственность нежелательна и должна быть устранена в продуктовой версии. Возможные пути устранения сводятся к двум: либо разворачивание полноценного сервиса в режиме отдельного процесса с HTTP-API и обращение к нему обоим веткам, либо приведение зависимостей к общему окружению, которое поддерживает обе архитектуры процессора. Выбор пути выходит за рамки текущей главы и должен быть согласован с эксплуатационными ограничениями целевой среды.

Интеграция в действующую образовательную платформу. Прототип реализован как самостоятельный компонент с унифицированным интерфейсом обращения к языковой модели и фиксированным форматом входных и выходных артефактов. Такой контур упрощает воспроизводимость экспериментов и не зависит от производственной инфраструктуры платформы. Полное сопряжение модуля с действующим сервисом «Прорыв» [30] — задача эксплуатационного уровня, требующая согласования внутренних форматов вектора компетенций, протокола вызова из боевого окружения, схемы хранения сгенерированных конспектов и регламента контроля качества. Эта задача относится к продуктовому развитию системы и не входит в задачи прототипной фазы. Архитектурно ничто не препятствует переходу: фиксированный JSON-формат описания сцены и единая точка входа `pipeline/run_unified.py` с подкомандами `conspect`, `geometry` и `auto` дают готовый контракт между прототипом и эксплуатационным окружением.

Отдельный рендерер числовой прямой. Конвертация типа `number_line` в сцену `function_plot` с одной горизонтальной прямой работоспособна, но визуально перегружает результат. Для числовых прямых следует ввести отдельную ветвь рендеринга, которая отрисовывает только горизонтальную ось с засечками и метками без второй оси и сетки. Это улучшение носит чисто эстетический характер, но существенно повлияет на восприятие иллюстраций к задачам

по уравнениям и неравенствам.

Объединение веток конспекта и пошаговой геометрии. В текущей версии модули `run_pipeline.py` и `geometry_auto.py` существуют как независимые ветви и не имеют общего входного интерфейса. Это удобно для разработки, поскольку позволяет развивать ветки независимо, но затрудняет ситуацию, когда учебный конспект включает геометрическую задачу с пошаговым разбором. Логически это решается тем, что планировщик конспекта в случае обнаружения геометрической задачи делегирует обработку модулю пошаговой геометрии и встраивает её результат в общий поток. Реализация такой делегации требует расширения формата плана конспекта и согласованного формата возврата результатов из геометрической ветки.

Расширение типологии визуалов. Тринадцать педагогических категорий, использованных в планировщике, охватывают типичные сценарии конспекта по математике, но не исчерпывают возможные форматы. Наблюдения за прогонами указывают на то, что у учеников встречаются запросы на иллюстрации векторных операций, на анимации стереометрических построений с поворотом тела, на изображения с подвижными точками. Каждое такое расширение требует одновременной поддержки в планировщике, конвертере и движке. При этом архитектура с промежуточным форматом упрощает добавление: новый тип сцены вводится только в одном месте, остальные модули получают изменения локально.

Адаптация визуальной сложности под уровень ученика. Текстовая часть конспекта в действующей версии уже адаптируется под вектор компетенций ученика: уровень детализации, число промежуточных шагов рассуждения и плотность примеров изменяются в зависимости от профиля. Визуальная часть в прототипной фазе таким параметром не управляется: иллюстрации формируются по фиксированной педагогической логике планировщика и не различают аудиторию. Содержательное направление развития — добавление в системный промпт планировщика и в конвертер дополнительного параметра, который задаёт желаемый уровень детализации иллюстрации. Для слабее подготовленного ученика этот параметр должен увеличивать число пояснительных подписей, выделять опорные элементы и предписывать пошаговое раскрытие построения. Для более подготовленного — наоборот, требовать лаконичного изображения с фиксацией только ключевых отношений. Реализация затрагивает только планировщик и конвертер и не требует изменений в формате сцены и движке. Качественная проверка такого механизма требует экспертной разметки иллюстра-

ций по уровню сложности и сопоставления её с заявленным целевым уровнем, что относится к ближайшим задачам исследования.

Эксперимент с моделями разной размерности. В прототипе используются две модели: `Qwen2.5-7B-Instruct-Turbo` в планировщике конспекта и `Llama-3.3-70B-Instruct-Turbo` в пошаговой геометрии. Соответствующий выбор обоснован прагматически. Содержательно стоит провести систематический эксперимент с моделями разной размерности и архитектуры (например, дополнительно с `Mixtral-8x7B`, `DeepSeek-Coder-V2`, рассуждающими моделями типа `o1`), чтобы выявить, какая часть результата зависит от размера модели, какая от способа обращения к ней, а какая от структуры самого контура. Такой эксперимент требует ресурсов API и формального протокола сравнения.

Обобщение на другие предметы ЕГЭ. Темой настоящей работы заявлены визуализации в учебных конспектах по предметам профильного ЕГЭ, тогда как реализованный прототип ограничен математикой. Такое сужение принято осознанно и связано не с архитектурными ограничениями контура, а с управляемым объёмом первой итерации: для отладки двухшаговой схемы и оценки её свойств требуется однородный корпус с устоявшейся типологией визуалов. Архитектура прототипа — двухшаговая декомпозиция с фиксированным контрактом между планировщиком и движком — спроектирована так, что расширение на другие предметные области затрагивает только две из четырёх её ступеней. Меняются педагогическая таксономия типов визуалов в планировщике и стратегии конвертации этих типов в сцены. Промежуточный JSON-формат описания сцены, движок рендеринга со схемами `Pydantic` и общая схема прогона остаются неизменными. Конкретные направления обобщения удобно наметить по характеру визуальных нагрузок, типичных для каждого предмета.

Физика. Учебные материалы по этому предмету предполагают новые классы изображений: схемы электрических цепей, диаграммы сил и моментов, ход лучей в геометрической оптике, графики зависимостей в кинематике и термодинамике. В планировщике появляются новые педагогические типы (условные обозначения — `circuit_diagram`, `force_diagram`, `ray_diagram`, `kinematics_graph`). Движку рендеринга требуется библиотека соответствующих примитивов — резисторов, источников ЭДС, векторов сил, тонких линз. Однако сама семантическая модель сцены остаётся прежней: точки, отрезки, ограничения и аннотации. Часть типов (графики и диаграммы) укладывается в уже существующие сцены `function_plot` и `diagram` без расширения движка.

Химия. Здесь основу визуальной нагрузки составляют структурные формулы моле-

кул, схемы реакций, диаграммы фазовых переходов и графики растворимости. Семейство сцен пополняется типом `molecular_structure`. При этом существующий решатель геометрических ограничений в принципе подходит для расчёта пространственного положения атомов по списку связей: задача выбора координат по ограничениям длин и углов связей структурно близка к уже поддерживаемым геометрическим построениям. Дополнительная сложность связана с условностями отображения химических связей (одинарных, двойных, ароматических колец), но это уровень примитивов рендеринга, а не архитектуры контура.

Биология. В материалах ЕГЭ по биологии преобладают схемы биологических процессов (фотосинтез, нервная регуляция, экологические пирамиды), классификации в виде дерева и сравнительные таблицы. Большая часть таких иллюстраций уже охватывается имеющимся типом сцены `diagram`: добавляются педагогические подтипы планировщика (`process_diagram`, `classification_tree`, `food_chain`), тогда как движок диаграмм не требует изменений. Это самый дешёвый с точки зрения инженерных затрат сценарий расширения.

Информатика и обществознание. Информатика естественным образом ложится на существующий движок: блок-схемы алгоритмов, диаграммы состояний конечных автоматов и графы зависимостей описываются в терминах сцены `diagram` с минимальными добавлениями примитивов узлов и стрелок. Обществознание требует преимущественно сравнительных таблиц, опорных конспектов и схем взаимосвязей институтов и понятий, которые также реализуются на существующем движке диаграмм. Для обоих предметов основная работа сосредоточена в планировщике и сводится к расширению педагогической таксономии и подбору соответствующих фрагментов системного промпта.

Во всех перечисленных направлениях содержательная часть расширения сводится к двум локализованным изменениям: пополнению педагогической таксономии планировщика и регистрации новой стратегии в конвертере. Промежуточный формат, движок рендеринга и протокол прогона остаются неизменными. Это и составляет основной инженерный аргумент в пользу выбранной двухшаговой архитектуры: она допускает развитие в сторону полноценной поддержки экзамена по всем профильным предметам без переписывания ядра контура. Систематическая оценка качества таких расширений требует отдельных предметных корпусов и согласованных с предметниками протоколов экспертной разметки и относится к задачам следующих итераций.

Контекст ограничений в практике учебных продуктов. Полученные результаты помещаются в более широкий ландшафт инструментов, которыми пользуются разработчики учебных продуктов. Готовые сервисы на базе вычислительных ядер, такие как Wolfram

Alpha, формируют визуализации высокого качества для аналитических задач, но не интегрированы в логику отдельного конспекта и не поддерживают пошаговое сопровождение в форме, привязанной к структуре учебного материала [28]. Диалоговые ассистенты на базе языковых моделей в составе образовательных платформ, такие как Khanmigo, делают акцент на голосовое и текстовое объяснение материала, но не претендуют на массовое автоматическое порождение векторных иллюстраций под конкретный текст [10]. Публикации в российском медиаполе, обсуждающие применение нейросетей в подготовке к ЕГЭ, фиксируют общественный запрос на автоматизированные инструменты сопровождения подготовки к экзамену, в том числе в части визуальных подсказок [31]. Описанный в данной главе прототип занимает нишу, не закрытую перечисленными решениями: автоматическое формирование иллюстраций для учебного материала по запросу. Его дальнейшее развитие может быть направлено на интеграцию с диалоговыми образовательными ассистентами и на сравнительные оценки качества с уже существующими сервисами.

3.13 Выводы по главе

В данной главе прототипная реализация двухшаговой системы генерации визуальных иллюстраций для конспектов ЕГЭ по математике рассмотрена с трёх точек зрения. С точки зрения архитектуры показано, что разделение на ступени планировщика, конвертера и движка с промежуточным форматом сцены позволяет изолировать ошибки разных уровней и получать содержательно разные классы наблюдений. Та же декомпозиция допускает расширение прототипа на другие предметы профильного ЕГЭ за счёт локальных изменений в планировщике и конвертере без переписывания ядра контура (подраздел 3.12). С точки зрения экспериментов на прототипном корпусе из четырёх конспектов и серии геометрических задач продемонстрирована работоспособность четырёхступенчатого контура (текст конспекта, план визуалов, описание сцены, файл SVG) на материалах профильного экзамена. С точки зрения итераций улучшений описаны тринадцать конкретных доработок, каждая из которых сужает класс наблюдавшихся сбоев и переносит часть рассуждения с языковой модели на детерминированные алгоритмы.

Ограниченность статистической базы и отсутствие формального протокола сравнения с базовой линией зафиксированы явно. Соответствующие задачи помечены как ближайшие шаги по программе работ. Их решение позволит превратить полученные качественные наблюдения в количественные оценки и поддержать заявленный архитектурный выбор более строгой эмпирической базой. Сводный анализ полученных результатов и их сопоставление с

целью и задачами введения вынесены в раздел *Заключение*.

Заключение

В настоящей работе разработан и реализован модуль автоматической генерации персонализированных учебных конспектов по математике ЕГЭ, центральным компонентом которого является пайплайн построения корректных математических иллюстраций по тексту. В данном заключении полученные результаты соотнесены с целью и задачами, сформулированными во введении, обозначены теоретическая и практическая значимость работы, а также зафиксированы её ограничения и направления дальнейшего развития.

Соответствие достигнутых результатов поставленным задачам. Цель работы — разработать и реализовать модуль генерации персонализированных учебных конспектов, включающих корректные визуальные материалы, — достигнута. Каждая из пяти задач, поставленных во введении, завершена в объёме, достаточном для прототипной демонстрации работоспособности контура и для перехода к стадии практического внедрения.

Первая задача — анализ существующих подходов к персонализации учебного контента, генерации учебных материалов и автоматическому построению математических визуализаций — решена в рамках обзора литературы и раздела об архитектуре системы. Показано, что среди существующих образовательных платформ ни одна не закрывает полный цикл от аналитики компетенций ученика к персонализированному конспекту с математическими иллюстрациями: одни системы реализуют адаптивный подбор задач без генерации объяснительного контента, другие работают в диалоговом формате без структурированных визуальных материалов. Показано также, что прямая генерация изображений моделями текста к изображению (text-to-image) систематически нарушает точные геометрические отношения и не обеспечивает корректного отображения математических подписей и формул. Это определило выбор двухшагового процесса с промежуточным структурированным представлением сцены.

Вторая задача — проектирование архитектуры процесса генерации конспектов от вектора компетенций ученика к структурированному тексту с визуальными элементами — завершена и закреплена в виде программного контура, описанного в подразделе 3.2. Контур разбит на четыре ступени с фиксированными контрактами по входу и выходу: разбиение конспекта на смысловые секции, двухпроходный планировщик визуалов, конвертер педагогической таксономии в технические сцены и движок векторного рендеринга. Такая декомпозиция обеспечивает локализуемость ошибок и независимое развитие ступеней, что подтверждено практикой итеративного улучшения прототипа (подраздел 3.10).

Третья задача — разработка схемы JSON-представления для математических визуализаций и реализация движка SVG-рендеринга на её основе — решена в составе модулей `pipeline/`. Введён универсальный формат сцены, поддерживающий три семейства иллюстраций (`function_plot`, `geometry`, `diagram`); реализован геометрический рендерер с солвером ограничений, обрабатывающим типовые построения школьной геометрии (медианы, высоты, биссектрисы, описанные и вписанные окружности, перпендикуляры, пересечения прямых, средние линии, касательные, сечения). Стереометрические сцены поддержаны через косую кабинетную проекцию с автоматической определёнкой видимых и скрытых рёбер по списку граней. Корректность рендеринга на типовых сценариях продемонстрирована примерами в подразделах 3.7 и 3.8, а также набором офлайн-тестов, не требующих обращения к внешнему API.

Четвёртая задача — интеграция модуля визуализаций в общий процесс генерации конспектов с учётом профиля ученика — решена в виде сквозного конвейера от исходного Markdown-конспекта до набора SVG с сохранением промежуточных артефактов на каждом этапе (подраздел 3.2). Дополнительно реализована единая точка входа `pipeline/run_unified.py` с подкомандами для конспектной и геометрической ветвей, что устраняет дублирование флагов и упрощает автоматизацию смешанных пакетных прогонов (см. итерацию 13 в табл. 3.8).

Пятая задача — оценка качества генерируемых конспектов — решена в объёме, соответствующем прототипной стадии. Проведены три независимых эксперимента с фиксированным протоколом и сохранением подробных метрик:

- 1 Серия повторных прогонов на корпусе из четырёх предметных конспектов (подраздел 3.7, табл. 3.3). Полная серия выявила содержательное наблюдение о повышенной дисперсии планировщика на конспекте по вероятности и подтвердила устойчивость показателей по токенам и времени в диапазоне 11–13 % относительно среднего.
- 2 Серия из трёх повторных прогонов на стабильном корпусе из восьми задач планиметрии и стереометрии (подраздел 3.11, табл. 3.10). Серия зафиксировала среднюю задержку полного прогона около 17 с и нулевой fix-rate по двадцати четырём ячейкам; на объединённой выборке из тридцати шести ячеек двух серий доля прогонов с инициированным повторным запросом составила приблизительно 0,028.
- 3 Сравнительный эксперимент двух языковых моделей разной размерности (`Llama-3.3-70B-Instruct` и `Qwen2.5-7B-Instruct-Turbo`) на одном корпусе при одинаковом протоколе (табл. 3.11). Эксперимент количественно подтвердил, что бо́льшая модель обеспечивает нулевой

fix-rate, тогда как меньшая модель показала fix-rate около 17 % и одну ячейку отказа из двадцати четырёх.

Дополнительно эмпирически подтверждён эффект промоутирования пост-валидатора визуализации ответа из режима диагностического предупреждения в режим замкнутого цикла исправления (табл. 3.9): доля ячеек с финальным замечанием снизилась с 5/9 до 0/9 при стоимости пяти дополнительных вызовов модели на девять прогонов и без структурных регрессов на принятых кандидатах.

Теоретическая и методологическая значимость. В работе систематизированы и обобщены аргументы в пользу двухшаговой схемы генерации математических иллюстраций с промежуточным структурированным представлением. Принятая декомпозиция, в которой языковая модель отвечает за семантику задачи, а специализированный движок — за геометрические и арифметические аспекты построения, согласуется с общей закономерностью, сформулированной по итогам тринадцати итеративных улучшений прототипа: устойчивость контура повышается тем сильнее, чем больше шагов рассуждения переносится с языковой модели на детерминированные алгоритмы. Эта закономерность имеет общий характер и применима к задачам структурированной генерации за пределами рассмотренной предметной области (например, к генерации диаграмм в химии, биологии, экономике), где требуется одновременно интерпретация естественной языковой постановки и точное построение по фиксированным правилам.

Дополнительный методологический вклад связан с введением замкнутого цикла программной верификации педагогического качества плана. Предложенная схема, в которой пост-валидатор по типу ожидаемого ответа (длина, высота, расстояние, угол, площадь, радиус, периметр, объём, доказательство, построение) проверяет наличие соответствующего геометрического объекта и инициирует адресный fix-запрос с защитой от тихих регрессов, представляет собой переносимый шаблон проектирования: предметная сторона валидации меняется, общая структура (детектор класса ответа, набор требований, контролируемый цикл с условием строгого улучшения) сохраняется. Соответствующий код реализован в функциях `_validate_answer_visualization` и `_build_answer_vis_fix_request` модуля `geometry_auto.py` и покрыт офлайн-тестами.

Практическая значимость. Реализованный модуль встраивается в действующую платформу подготовки к ЕГЭ «Прорыв» [30] в качестве компонента, отвечающего за автоматическое формирование персонализированных учебных материалов. Автоматизация полного

цикла «текст → план → сцена → SVG» устраняет необходимость ручного создания иллюстраций для каждого индивидуального профиля компетенций и делает практически достижимой генерацию учебных материалов под конкретного ученика в близком к реальному времени. Сохранение промежуточных артефактов в виде самостоятельных JSON-файлов на каждом этапе обеспечивает воспроизводимость прогонов и снижает стоимость диагностики типовых сбоев на порядок: характерные расследования сводятся к проверке одного промежуточного файла без повторных вызовов модели. Для геометрической ветви эта же практика позволяет отделить процесс отладки рендеринга от стоимости обращения к языковой модели и проводить инженерные эксперименты на сохранённых планах.

Архитектурно предложенный подход применим и за пределами математики ЕГЭ. Промежуточный JSON-формат и принцип разделения ответственности между планировщиком, конвертером и движком переносимы на любую предметную область, где требуется структурированная генерация графических иллюстраций под индивидуальный учебный контент.

Ограничения работы. Полученные количественные оценки следует читать как точечные выборочные оценки на прототипном корпусе, а не как окончательные показатели качества. Все интеграционные эксперименты проведены на корпусе ограниченного размера (четыре предметных конспекта и восемь геометрических задач), а формальная сравнительная оценка с альтернативной базовой линией — прямой генерацией SVG моделью текста к изображению — в настоящей редакции не проведена. Используемые в прототипе показатели имеют преимущественно технический характер (доля прогонов без ошибок, средняя задержка, число токенов, fix-rate), тогда как педагогическое качество иллюстраций требует независимой экспертной разметки силами учителей математики. Дополнительные ограничения связаны с двойственностью текущей реализации рендеринга: сосуществование полноценного сервиса `svg-generator` и встроенного запасного рендерера в `run_pipeline.py` обусловлено инфраструктурными причинами на конкретной рабочей машине и в продуктовой версии должно быть устранено. Перечисленные ограничения подробно зафиксированы в подразделе 3.12 и отделены от установленных результатов работы.

Направления развития. По итогам работы намечены следующие задачи ближайшего этапа, систематизированные в подразделе 3.12. Первая — расширение корпуса до десятков конспектов и сотен геометрических задач с проведением серии повторных прогонов и формированием статистически обеспеченных оценок. Вторая — проведение сравнительного эксперимента с базовой линией `text-to-image` по согласованным группам показателей (тех-

нические свойства прогона, геометрическая корректность объектов сцены, педагогическая информативность по протоколу независимой экспертной разметки). Третья — введение формального набора педагогических метрик и согласование протокола экспертной разметки с руководителем работы. Четвёртая — унификация рендеринга в единое окружение с устранением необходимости в запасном рендерере. Пятая — расширение типологии визуалов в направлении векторных операций, анимированных стереометрических построений и сцен с подвижными точками. Шестая — объединение конспектной и геометрической ветвей конвейера через делегирование пошаговой геометрии из планировщика конспекта при обнаружении соответствующих задач. Седьмая — систематический эксперимент с моделями разной размерности и архитектуры (включая рассуждающие модели) для количественного разнесения вклада размера модели, способа обращения к ней и структуры самого контура.

Итог. Поставленная во введении цель достигнута: разработан и реализован прототип модуля автоматической генерации персонализированных учебных конспектов с математическими иллюстрациями, обеспечивающий геометрическую корректность визуальных материалов и встраиваемый в действующую образовательную платформу. Результаты прототипа подтверждают работоспособность принятого архитектурного решения с двухшаговой схемой и промежуточным структурированным представлением сцены. Зафиксированные ограничения переведены в перечень задач ближайшего этапа, выполнение которых превратит полученные качественные наблюдения в количественные оценки и обеспечит более строгую эмпирическую базу для заявленных архитектурных выводов.

Список литературы

1. Luca Beurer-Kellner, Marc Fischer и Martin Vechev. “Prompting Is Programming: A Query Language for Large Language Models”. В: *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2023. DOI: [10.1145/3591300](https://doi.org/10.1145/3591300). arXiv: [2212.06094](https://arxiv.org/abs/2212.06094).
2. Benjamin S. Bloom. “The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring”. В: *Educational Researcher* 13.6 (1984), с. 4—16. DOI: [10.3102/0013189X013006004](https://doi.org/10.3102/0013189X013006004).
3. Ihab Bosheah и др. “Challenges in Generating Accurate Text in Images: A Benchmark for Text-to-Image Models”. В: *Applied Sciences* 15.5 (2025), с. 2274. DOI: [10.3390/app15052274](https://doi.org/10.3390/app15052274).
4. Carnegie Learning. *MATHia: AI-Powered Math Learning Software*. 2025. URL: <https://www.carnegielearning.com/solutions/math/mathia/> (дата обр. 05.04.2026).
5. Zhongzhou Chen, Chris Chudzicki, Daniel Palumbo, Giora Alexandron, Youn-Jeng Choi, Qian Zhou и David E. Pritchard. “Researching for Better Instructional Methods Using AB Experiments in MOOCs: Results and Challenges”. В: *Research and Practice in Technology Enhanced Learning* 11.1 (2016), с. 9. DOI: [10.1186/s41039-016-0034-4](https://doi.org/10.1186/s41039-016-0034-4).
6. Albert T. Corbett и John R. Anderson. “Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge”. В: *User Modeling and User-Adapted Interaction* 4.4 (1994), с. 253—278. DOI: [10.1007/BF01099821](https://doi.org/10.1007/BF01099821).
7. Aaditya Gupta, Jonas Reddig, Tyler Calo и др. “Beyond Final Answers: Evaluating Large Language Models for Math Tutoring”. В: (2025). arXiv: [2503.16460](https://arxiv.org/abs/2503.16460).
8. Owen Henkel и др. “Retrieval-Augmented Generation to Improve Math Question Answering”. В: *NeurIPS Workshop on Generative AI for Education*. 2024. URL: <https://files.eric.ed.gov/fulltext/ED675655.pdf>.
9. Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin и Ting Liu. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. В: (2023). arXiv: [2311.05232](https://arxiv.org/abs/2311.05232).
10. Khan Academy. *Khanmigo — AI-Powered Learning*. 2026. URL: <https://www.khanmigo.ai/> (дата обр. 05.04.2026).

11. Ron Kohavi, Roger Longbotham, Dan Sommerfield и Randal M. Henne. “Controlled Experiments on the Web: Survey and Practical Guide”. B: *Data Mining and Knowledge Discovery* 18.1 (2009), с. 140—181. DOI: [10.1007/s10618-008-0114-1](https://doi.org/10.1007/s10618-008-0114-1).
12. Chris van der Lee, Albert Gatt, Emiel van Miltenburg и Emiel Krahmer. “Best Practices for the Human Evaluation of Automatically Generated Text”. B: *Proceedings of the 12th International Conference on Natural Language Generation (INLG)*. 2019, с. 355—368. URL: <https://aclanthology.org/W19-8643>.
13. Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel и Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. B: *Advances in Neural Information Processing Systems (NeurIPS)*. T. 33. 2020. arXiv: [2005.11401](https://arxiv.org/abs/2005.11401).
14. Yanyan Li и др. “Retrieval-Augmented Generation for Educational Application”. B: *Computers and Education: Artificial Intelligence* (2025). URL: <https://www.sciencedirect.com/science/article/pii/S2666920X25000578>.
15. Jialong Liang и др. “Controllable Text Generation for Large Language Models: A Survey”. B: (2024). arXiv: [2408.12599](https://arxiv.org/abs/2408.12599).
16. Yu Lu и др. “Advanced Knowledge Tracing”. B: *Journal of Educational Data Mining* (2024). URL: <https://jedm.educationaldatamining.org/index.php/JEDM/article/download/689/215>.
17. McGraw Hill. *ALEKS: Assessment and Learning in Knowledge Spaces*. 2025. URL: <https://www.aleks.com/> (дата обр. 05.04.2026).
18. Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J. Guibas и Jascha Sohl-Dickstein. “Deep Knowledge Tracing”. B: *Advances in Neural Information Processing Systems (NeurIPS)*. T. 28. 2015. arXiv: [1506.05908](https://arxiv.org/abs/1506.05908).
19. Johanna Schoenherr и Stanislaw Schoenherr. “Learning with Visualizations Helps: A Meta-Analysis of Visualization Interventions in Mathematics Education”. B: *Educational Research Review* 44 (2024), с. 100639. DOI: [10.1016/j.edurev.2024.100639](https://doi.org/10.1016/j.edurev.2024.100639).
20. Xiaoyu Song, Jiahao Li, Yan Tang, Tao Zhao, Yuanfang Chen и Ziwen Guan. “Knowledge Tracing: A Survey”. B: *ACM Computing Surveys* 55.11 (2022), с. 1—37. DOI: [10.1145/3569576](https://doi.org/10.1145/3569576).

21. Saiying Steenberg-Hu и Harris Cooper. “A Meta-Analysis of the Effectiveness of Intelligent Tutoring Systems on K–12 Students’ Mathematical Learning”. В: *Journal of Educational Psychology* 105.4 (2013), с. 970—987. DOI: [10.1037/a0032447](https://doi.org/10.1037/a0032447).
22. Shengze Tan и др. “Artificial Intelligence-Enabled Adaptive Learning Platforms”. В: *Computers and Education: Artificial Intelligence* (2025). URL: <https://www.sciencedirect.com/science/article/pii/S2666920X25000694>.
23. Kurt VanLehn. “The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems”. В: *Educational Psychologist* 46.4 (2011), с. 197—221. DOI: [10.1080/00461520.2011.611369](https://doi.org/10.1080/00461520.2011.611369).
24. W3C. *Scalable Vector Graphics (SVG) 2*. W3C Candidate Recommendation. 2018. URL: <https://www.w3.org/TR/SVG2/>.
25. Jiaqi Wang и др. “MagicGeo: Training-Free Text-Guided Geometric Diagram Generation”. В: (2025). arXiv: [2502.13855](https://arxiv.org/abs/2502.13855).
26. Jingxuan Wei и др. “From Words to Structured Visuals: A Benchmark and Framework for Text-to-Diagram Generation”. В: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025. arXiv: [2411.11916](https://arxiv.org/abs/2411.11916).
27. Brandon T. Willard и Rémi Louf. “Efficient Guided Generation for Large Language Models”. В: (2023). arXiv: [2307.09702](https://arxiv.org/abs/2307.09702).
28. Wolfram Research. *Wolfram/Alpha: Computational Intelligence*. 2025. URL: <https://www.wolframalpha.com/> (дата обр. 05.04.2026).
29. Hanyi Xu и др. “Large Language Models for Education: A Survey”. В: (2024). arXiv: [2405.13001](https://arxiv.org/abs/2405.13001).
30. Прорыв. *Подготовка к ЕГЭ*. 2026. URL: <https://exambreakout.ru/> (дата обр. 05.04.2026).
31. РБК Тренды. *Искусственный интеллект в ЕГЭ: как нейросети пишут и проверяют экзамены*. 2024. URL: <https://trends.rbc.ru/trends/innovation/6544ad2d9a79474f5df5fd> (дата обр. 05.04.2026).
32. РЕШУ ЕГЭ. *Образовательный портал для подготовки к экзаменам*. 2025. URL: <https://ege.sdmgia.ru/> (дата обр. 05.04.2026).
33. Умскул. *Подготовка к экзаменам*. 2026. URL: <https://umschool.net/> (дата обр. 05.04.2026).

34. Учи.ру. *Интерактивная образовательная платформа*. 2025. URL: <https://uchi.ru/> (дата обр. 05.04.2026).
35. ФИПИ. *Демоверсии, спецификации, кодификаторы ЕГЭ*. 2026. URL: [https://fipi.ru/ ege/demoversii-specifikacii-kodifikatory](https://fipi.ru/ege/demoversii-specifikacii-kodifikatory) (дата обр. 05.04.2026).
36. ФИПИ. *Открытый банк заданий ЕГЭ*. 2026. URL: [https://fipi.ru/ ege/otkrytyy-bank-zadaniy-ege](https://fipi.ru/ege/otkrytyy-bank-zadaniy-ege) (дата обр. 05.04.2026).
37. Фоксфорд. *Подготовка к ЕГЭ по математике*. 2026. URL: [https://foxford.ru/ catalog/courses/matematika/ege](https://foxford.ru/catalog/courses/matematika/ege) (дата обр. 05.04.2026).
38. Школково. *Подготовка к ЕГЭ/ОГЭ*. 2026. URL: <https://3.shkolково.online/> (дата обр. 05.04.2026).
39. Яндекс Учебник. *Репетитор AI по математике*. 2026. URL: [https://yandex.ru/ support/education-students/ru/common-concepts/ai-tutor](https://yandex.ru/support/education-students/ru/common-concepts/ai-tutor) (дата обр. 05.04.2026).